



中南大學
CENTRAL SOUTH UNIVERSITY



机电工程学院

智能健康监测手表设计说明书

姓名	班级	学号
王子杰	机械 2304 班	8203230717

指导教师 张友旺

学院 机电工程学院

2026 年 6 月 28 日

摘要

随着人口老龄化加剧与慢性病管理需求提升，可穿戴健康监测设备成为消费电子与远程医疗的关键融合载体。本文基于意法半导体 STM32F103ZE 微控制器与 FreeRTOS（Real-Time Operating System）实时操作系统，设计并实现了一套开源智能健康监测手表系统，集成心率血氧检测、计步与运动状态识别、体温估算、实时时钟、OLED 显示、蓝牙通信及离线语音控制九大核心功能。

硬件采用双 I²C（Inter-Integrated Circuit）总线架构：I²C1 经 GPIO 重映射，对接 MAX30102 心率血氧传感器与 MPU6050 六轴姿态传感器；I²C2 独立驱动 SSD1306 OLED 显示屏。双总线物理隔离 PPG 高速采样数据流与屏幕刷新数据流，有效规避总线带宽竞争引发的显示卡顿、数据采样丢失等问题。软件采用四层分层架构，依次为应用层、算法层、驱动层与硬件层，分别负责任务调度与业务处理、信号运算与算法补偿、传感器驱动适配、总线时序封装与异常恢复，各层单向依赖、接口规范，可维护性与可移植性良好。

核心算法方面，系统采用 0.5Hz 一阶 IIR 高通与 5Hz 一阶 IIR 低通级联滤波，滤除 PPG 信号基线漂移与高频噪声，保留有效脉搏波信息；通过动态阈值峰值检测提取脉搏波峰，结合峰间间隔计算瞬时心率，并采用平滑系数 0.15 的 EMA 算法优化心率数据。血氧检测基于双波长朗伯比尔定律，通过红外与红光 PPG 信号的 AC/DC 比值计算特征参数，代入校准公式完成血氧估算，结果经限幅平滑后输出。计步算法通过加速度数据剔除重力分量，搭配自适应阈值与最小步间隔约束，结合步频、静止状态实现运动状态识别。体温检测依托 DS18B20 采集皮肤温度，结合 MPU6050 片内环境温度进行线性补偿，估算人体核心体温。

系统基于 FreeRTOS 完成多任务可靠性优化，通过互斥锁保护 I²C 总线访问、临界区保障全局数据原子读取，搭配 4s 超时独立看门狗，解决总线死锁、任务挂死问题。同时针对传感器 FIFO 读取、异步温度转换、屏幕脏页刷新、串口溢出报错等工程问题做专项优化，保障设备长期稳定运行。

实测结果表明：心率测量误差 $\leq \pm 5$ BPM，血氧测量误差 $\leq \pm 3\%$ ，百步计数平均误差 2.3 步，体温测量误差 $\leq \pm 0.8$ °C，设备可连续稳定运行 4 小时无故障。本系统功能完善、架构合理，可为后续低功耗优化、可视化开发提供可靠的软硬件基础。

关键词：STM32F103ZE；FreeRTOS；光电体积描记术（PPG）；血氧饱和度；自适应计步；温度补偿；可穿戴健康监测；实时操作系统；I²C 总线；独立看门狗

目录

1 绪论	4
1.1 课题研究背景及意义	4
1.2 国内外健康监测手表研究现状	5
1.3 课题主要研究内容与文档结构安排	5
1.4 本设计的特色与创新点	6
2 系统总体设计	7
2.1 系统功能与性能需求分析	7
2.2 系统整体软硬件总体设计方案	9
2.3 核心硬件选型与关键参数	10
2.4 系统引脚分配与接口规划	12
2.5 电源与时钟设计方案	13
2.6 通信协议选型对比	14
2.7 软件开发环境与工具链	14
3 软件设计与实现	15
3.1 四层软件架构设计	15
3.2 FreeRTOS 多任务调度设计	17
3.3 硬件抽象层设计：I ² C 与 UART	21
3.4 传感器驱动层设计	24
3.5 核心算法层设计	28
3.6 蓝牙通信与语音控制设计	37
3.7 线程安全与并发控制	38
3.8 独立看门狗与故障恢复	39
3.9 DWT 时间基准与精确延时	39
3.10 内存布局与算法静态数组设计	40
3.11 系统初始化顺序设计	41
3.12 错误处理与容错设计	42
3.13 关键调用链与数据流分析	42
4 系统硬件实现与调试	44
4.1 硬件集成方案与上电检查	44
4.2 接口电路设计	44
4.3 Keil 工程配置	45
4.4 系统调试方法	46
4.5 系统调试方法与常见问题	47
4.6 电磁兼容与抗干扰设计	47
5 系统测试与结果分析	48
5.1 系统分项测试	48
5.2 测试数据与系统性能分析	50
5.3 功耗与可靠性分析	51
5.4 算法参数对性能的影响分析	51
5.5 与同类开源项目的对比	52
6 总结与展望	53
6.1 课题整体工作总结	53
6.2 系统优化与功能拓展展望	53

附录 A 核心配置文件说明	55
A.1 algorithms.h 算法参数配置	55
A.2 FreeRTOSConfig.h 内核配置	56
A.3 传感器头文件配置	56
A.4 main.c 初始化流程与异常钩子	56
附录 C 术语表	57
致谢	57
参考文献	58

1 绪论

1.1 课题研究背景及意义

随着社会经济持续发展与居民生活方式的转变，健康管理正由被动式疾病治疗向主动式健康监测加速演进。据世界卫生组织（WHO）发布的统计报告，心脑血管疾病已跃居全球首位致死原因，每年造成约 1790 万人死亡，其中相当比例的猝死事件在发病前缺乏连续的生理参数记录。与此同时，我国 60 岁以上人口占比持续攀升，高血压、糖尿病、慢性阻塞性肺病等慢性病的长期管理需求日益迫切。在此背景下，能够对心率、血氧饱和度（ SpO_2 ）、体温、活动量等关键生理参数进行连续、无创、便携监测的可穿戴健康设备，兼具重要的社会价值。

近年来，以 Apple Watch、华为手环、小米手环为代表的消费级可穿戴设备迅速普及，推动了健康监测技术向民用化方向发展。此类设备普遍集成了多种微型传感器，借助嵌入式系统对生理信号进行实时采集与处理，为用户提供直观的健康数据参考。然而，现有商用产品在工程实践中仍存在三方面局限：其一，核心算法以二进制形式封闭，使用者难以理解其信号处理与生理建模过程；其二，数据接口多为私有协议，难以与第三方科研平台对接；其三，二次开发门槛较高，不利于在嵌入式教学与开源科研中深入复用。上述局限在一定程度上制约了健康监测底层技术在高校教学与科研领域的传播。

本设计以 STM32F103ZE 微控制器为硬件核心，以 FreeRTOS 实时操作系统为软件内核，自主设计并实现了一套开源的智能健康监测手表系统。系统集成了 MAX30102 心率血氧传感器、MPU6050 六轴加速度传感器、DS18B20 体温传感器、DS1302 实时时钟模块、SSD1306 OLED 显示屏、HC-05 蓝牙模块及 ASR PRO 离线语音识别模块，覆盖了心率监测、血氧检测、步数计数、运动状态识别、体温测量、实时时钟、OLED 显示、蓝牙通信与语音控制等多项功能，形成了一条从底层传感器驱动到上层算法处理的全链路技术路径。

本设计的研究意义可归纳为以下三个方面。第一，从底层硬件驱动到上层算法处理均实现全链路自主研发，具有完整的知识产权与教学开放性，可作为嵌入式系统课程的综合实践载体。第二，基于 FreeRTOS 的多任务软件架构充分体现了抢占式调度、任务同步互斥、优先级反转避免等实时系统设计理念，为嵌入式实时操作系统教学提供了一个结构完整、可运行可调试的实践案例。第三，PPG 信号处理、自适应阈

值计步、环境温度补偿等算法均具有扎实的理论基础与明确的工程可实现性，相关公式与参数全部公开，可为可穿戴健康监测领域的进一步研究提供可复用的技术参考。

1.2 国内外健康监测手表研究现状

在消费电子领域，Apple Watch 自 2015 年发布以来已迭代至多代产品。Apple Watch Series 4 及以上型号集成了通过美国 FDA 认证的 ECG 心电图功能，其心率监测基于光电体积描记术（Photoplethysmography, PPG）技术，通过绿色 LED 与红外 LED 组合照射皮肤，利用光电二极管检测血流容积变化引起的吸光率波动。华为手环系列采用自研 TruSeen 技术，结合多路 PPG 信号采集与人工智能算法，在运动状态下仍能保持较高的心率测量准确率。Fitbit、Garmin 等品牌则侧重于运动健康追踪，提供完善的计步、睡眠分期与卡路里消耗估算功能。上述产品的共同特征是：传感器高度集成化、算法高度定制化、用户体验高度场景化，但其内部实现细节对外部研究者基本封闭。

在学术研究领域，PPG 信号处理算法的探索持续活跃。基于时域分析的自适应阈值峰值检测方法因计算量小、实时性强、对处理器资源要求低，非常适合资源受限的嵌入式平台，是当前 MCU 级可穿戴设备的主流方案。频域分析方法（如快速傅里叶变换 FFT）虽抗工频干扰能力较强，但对处理器运算能力与样本窗口长度要求较高，在低主频 MCU 上难以满足实时性。近年来，深度学习在 PPG 信号处理中的应用日益受到关注，卷积神经网络与长短期记忆网络在运动伪影抑制与房颤筛查等任务上表现出色，但神经网络模型在 MCU 平台上的部署仍面临 Flash 占用大、推理延迟高、能耗上升等挑战。在血氧检测方面，基于双波长 Lambert-Beer 定律的 R-ratio 方法因其物理意义明确、实现简便，是目前最成熟、应用最广泛的方案，几乎所有商用脉搏血氧仪均采用该原理。

从技术角度看，在 MCU 平台上实现可穿戴健康监测面临三大挑战。一是多传感器并发采集与实时处理的资源受限问题，需要在 72 MHz 主频与 64 KB RAM 的约束下完成 PPG 滤波、计步、显示刷新与蓝牙通信。二是运动伪影与个体差异对算法精度的影响，需要设计鲁棒的信号处理与自适应算法。三是长期佩戴的可靠性与功耗问题，需要通过合理的任务调度、错误恢复与低功耗设计保障系统稳定运行。本系统的设计与实现正是围绕这些挑战展开的。

1.3 课题主要研究内容与文档结构安排

本设计的主要研究内容可归纳为五个方面。其一，系统硬件方案设计，包括主控芯片选型、各传感器模块选型与接口电路设计、双 I²C 总线架构设计、电源管理方案设计。其二，底层驱动程序开发，涵盖 I²C、UART、1-Wire 及 GPIO 模拟三线时序等通信协议的软件实现，以及各传感器初始化与数据读取函数的编写，并集成超时保护、错误恢复与互斥锁机制。其三，核心算法设计与实现，包括基于 PPG 信号的心率检测算法、基于 R-ratio 的血氧饱和度估算算法、基于加速度幅值分析的自适应阈值计步算法，以及基于环境温度参考的体温补偿算法。其四，基于 FreeRTOS 的嵌入式软件架构设计，包括多任务划分、优先级分配、任务间通信机制与线程安全保护设计。其五，系统集成测试与性能分析，通过分项测试与整机连续运行测试验证系统功能与可靠性。

1.4 本设计的特色与创新点

与现有开源健康监测项目相比，本设计在以下方面具有鲜明特色与创新价值。

全链路自主研发与教学开放性。 本系统从底层 GPIO 配置、I²C/UART/1-Wire 驱动到 PPG 算法、FreeRTOS 任务调度均自行实现，未依赖现成传感器库或算法库。所有源代码、算法参数、引脚定义、通信协议均公开，便于教学人员逐行讲解与学生深入理解。

双 I²C 总线架构的工程化设计。 针对 MAX30102 高采样率与 OLED 大刷新带宽的潜在冲突，本系统采用 I2C1 承载传感器、I2C2 独立承载显示屏的方案。这一设计看似简单，却是保障 PPG 数据连续性与显示实时性的关键，体现了从需求出发的总线规划思想。

多层容错与故障自恢复机制。 系统在 I²C 错误时自动复位总线、在 UART overrun 时自动清除错误、在断言失败时输出诊断信息、在任务挂死时由 IWDG 复位。这些机制使系统在面对硬件接触不良、电磁干扰、软件异常时具备较强的鲁棒性，适合可穿戴设备长期佩戴场景。

算法参数集中配置与可解释性。 所有算法可调参数集中在 `algorithms.h` 中，并附带物理意义注释。学生或开发者无需阅读算法实现即可调整 PPG 滤波截止频率、峰值检测灵敏度、计步阈值、温度补偿系数等，这大大降低了二次开发门槛。

多模态人机交互。 系统同时支持按键翻页、蓝牙命令与语音控制三种交互方式，覆盖了可穿戴设备常见的交互场景。ASR PRO 离线语音识别避免了云端依赖，保护了用户隐私，响应速度也更快。

2 系统总体设计

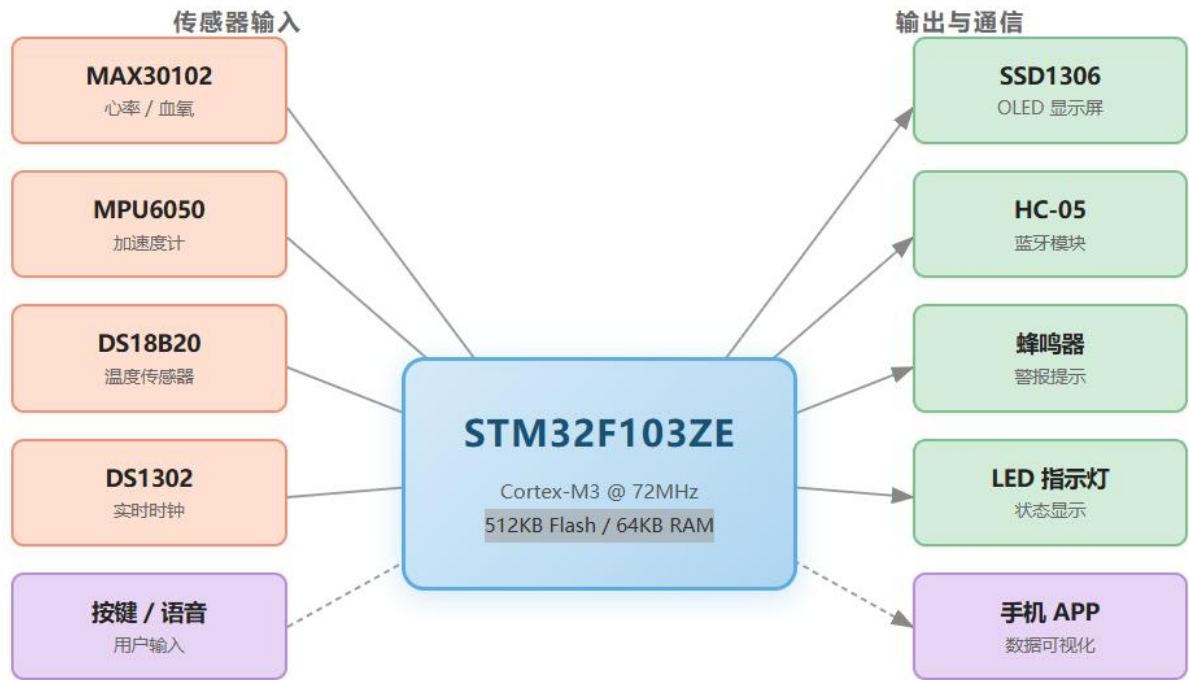


图 1-1 系统总体框图

2.1 系统功能与性能需求分析

本系统定位为一款基于嵌入式平台的开源智能健康监测手表，主要面向嵌入式系统教学演示与个人健康监测场景。综合可穿戴设备的使用特征与教学示范需求，系统需在有限算力与功耗约束下实现以下八项核心功能。

（1）心率实时监测。通过 MAX30102 PPG 传感器以 100 Hz 采样率采集红外光反射信号，经一阶 IIR 带通滤波去除直流分量与高频噪声后，采用动态阈值峰值检测算法提取脉搏波峰，由峰间间隔计算瞬时心率，并以指数移动平均（EMA）对心率值进行平滑处理。心率测量范围应覆盖 30~200 BPM，首次锁定时间不超过 8 秒，刷新周期不超过 2 秒。

（2）血氧饱和度检测。同步采集红外光与红光两路 PPG 信号，分别计算其交流分量与直流分量之比，进一步计算双波长 R-ratio 参数，代入经验校准公式估算血氧值。测量范围限定在 50%~100%，刷新周期不超过 5 秒，校准参数支持通过蓝牙指令远程调整。

（3）步数计数与运动状态识别。通过 MPU6050 加速度计获取三轴加速度，计算合加速度并分离重力分量，采用自适应阈值检测步态峰值，结合最小步间隔与步态连续性约束抑制误触发，同时根据步频与空闲时间将活动状态区分为静止、步行、跑步

与晃动四类。最小步间隔设为 300 ms 以抑制重复计数（兼顾跑步场景），同时设自适应阈值下限 STEP_MIN_THRESH = 200 抗静止噪声。

（4）体温测量与补偿。通过 DS18B20 数字温度传感器测量皮肤温度，结合 MPU6050 片内温度传感器提供的环境温度参考值，利用线性补偿公式估算核心体温。DS18B20 配置为 12 位分辨率，标称精度 $\pm 0.5\text{ }^{\circ}\text{C}$ ，温度读取周期约 800 ms，当前版本采用数值限幅与平滑保障可靠性，预留 CRC 校验接口。

（5）实时时钟。通过 DS1302 RTC 模块提供精确的年月日时分秒信息，支持通过蓝牙指令远程校准；模块内置 CR2032 纽扣电池，可在系统主电源断开后维持时钟连续运行，避免掉电走时丢失。

（6）OLED 数据显示。采用 0.96 寸 SSD1306 OLED 显示屏，以五页面轮播方式分别显示心率、血氧、步数、体温与时间；支持按键翻页与语音命令切换两种交互方式，自动刷新周期 500 ms，采用脏页刷新机制降低 I²C 总线占用。

（7）蓝牙无线通信。通过 HC-05 蓝牙模块，默认每 10 秒自动推送一条 JSON 格式的健康数据包至手机端（支持 2~10 秒动态配置）；同时支持接收 TIME 命令进行时间校准、CAL 命令调整血氧校准参数，命令以换行符分隔。

（8）语音控制。通过 ASR PRO 离线语音识别模块识别 9 种预设语音命令，包括页面切换、数据显示选择、步数清零等操作，响应时间不超过 100 ms，单字节命令码经 UART2 上报。

在系统性能方面，综合上述功能需求，需满足表 2-1 所列关键指标。系统应能在常温环境下连续稳定运行 4 小时以上，无栈溢出、无死机、无数据异常，并具备基本的错误自恢复能力（如 I²C 总线死锁后自动复位）。

表 2-1 系统关键性能指标需求

指标项	设计要求	说明
PPG 采样率	100 Hz	MAX30102 SpO2 模式配置
心率范围	30~200 BPM	覆盖静息至剧烈运动
心率刷新周期	$\leq 2\text{ s}$	EMA 平滑后输出
血氧范围	50%~100%	超限裁剪并平滑
OLED 刷新周期	500 ms	脏页刷新
蓝牙上报间隔	2 s	JSON 数据包
温度读取周期	$\approx 800\text{ ms}$	DS18B20 12 位模式
计步最小步间隔	300 ms	抑制重复计数

除了上述功能与性能指标，可穿戴设备还面临体积、功耗、佩戴舒适性与环境适应性等约束。本系统采用模块化传感器方案，各模块通过排针或杜邦线连接，便于在原型阶段快速验证。后续产品化时可进一步整合为定制 PCB，减小体积并优化电源管理。

2.2 系统整体软硬件总体设计方案

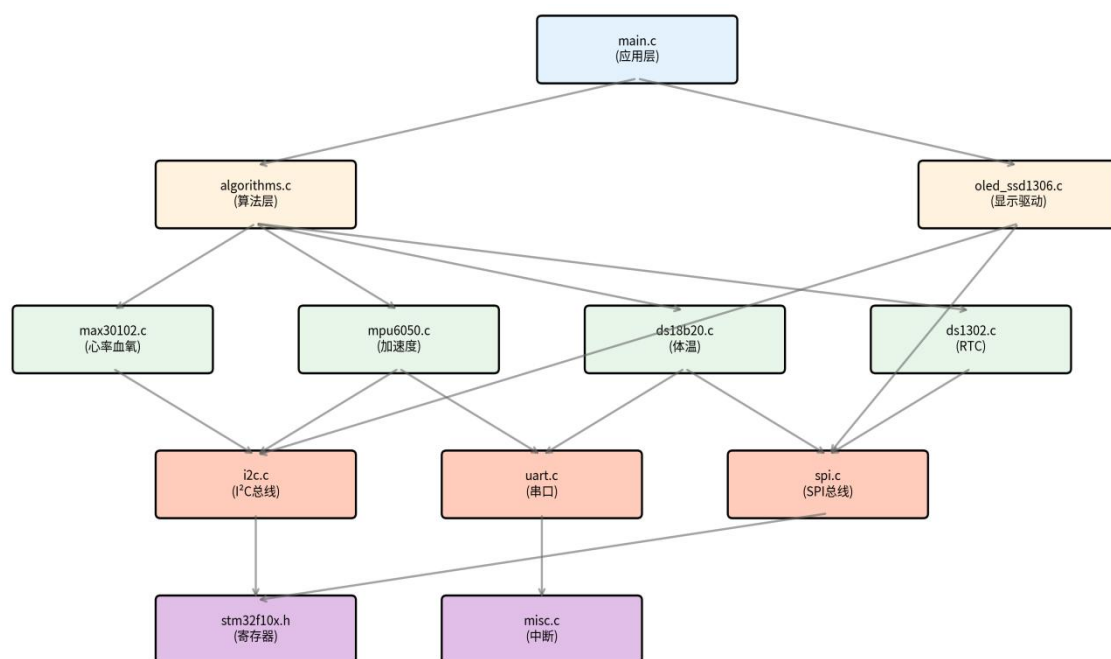


图 2-2 模块依赖关系图

本系统采用模块化分层设计思想，将系统划分为硬件层、驱动层、算法层与应用层四个层次，各层职责清晰、接口明确，遵循单向依赖原则，即上层仅可调用相邻下层接口，不允许跨层访问或同层相互依赖。该设计有效降低了模块间耦合度，提升了代码的可维护性与可移植性，也为后续平台迁移（如更换主控或传感器型号）预留了清晰的替换边界。

在硬件层，STM32F103ZE 微控制器作为核心处理单元，通过两组 I²C 总线与外部传感器和显示模块通信。I²C1 总线（重映射至 PB8=SCK、PB9=SDA）连接 MAX30102 心率血氧传感器与 MPU6050 六轴加速度传感器；I²C2 总线（PB10=SCK、PB11=SDA）独立连接 SSD1306 OLED 显示屏。DS18B20 温度传感器通过单总线（1-Wire）协议连接至 PA1；DS1302 实时时钟通过三线 SPI-like 时序连接至 PB1（CLK）、PB13（CE）、PB14（DAT）；HC-05 蓝牙模块通过 USART1（PA9=TX、PA10=RX）与主控通信；ASR PRO 语音模块通过 USART2（PA2=TX、PA3=RX）与主控通信。用户按键使用 PA0（WK_UP），配合开漏输出与强下拉实现低电平释放、

高电平触发。

驱动层封装了各硬件模块的初始化与数据读写操作，为上层提供统一的函数调用接口。每个传感器模块遵循 `Init()` + `ReadData()` 的统一接口规范，`Init()` 返回 0 表示初始化成功、负数表示错误码，使上层可通过返回值统一判断器件在线状态。I²C 与 UART 通信协议进一步被抽象为硬件抽象层（HAL），提供寄存器级的读写封装，并集成超时保护、错误恢复与 FreeRTOS 互斥锁等机制，确保多任务环境下的线程安全。

算法层负责所有信号处理与数据融合计算，包含 PPG 信号处理流水线（一阶 IIR 带通滤波、动态阈值峰值检测、心率与血氧计算）、计步算法（重力分离、自适应阈值、步频计算）以及体温补偿算法。算法层不直接访问任何硬件寄存器，所有输入数据以函数参数或全局变量形式传入，具有完全的硬件无关性，便于单元测试与跨平台移植。

分层架构使各层职责清晰，上层无需关心底层硬件实现。当更换传感器时，仅需修改驱动层；更换主控平台时，仅需重新实现 HAL 层（`i2c.c`、`uart.c` 等）；算法层能够脱离硬件独立测试，提高了系统的可维护性、可移植性和开发效率。

层间接口约定。 本系统约定：应用层调用算法层的函数（如 `PPG_ProcessSamples()`、`Step_ProcessAccel()`、`Compensate_Temperature()`）与驱动层的 `Init()/ReadData()` 函数；算法层调用驱动层的 `ReadData()` 函数获取数据，但不直接调用 HAL 层；驱动层调用 HAL 层的 `I2C_ReadReg()`、`I2C_WriteReg()`、`UART1_Send()` 等函数；HAL 层调用 STM32 StdPeriph 库或直接访问寄存器。任何跨层调用（如应用层直接访问 I²C 寄存器）都被视为违反架构约定，应予以避免。

数据流向。 传感器原始数据从硬件层向上流动：硬件 → HAL → 驱动 → 算法。处理后的健康数据通过全局变量供应用层读取：算法 → 全局变量 → 应用层任务。控制命令从应用层向下流动：应用层 → 驱动/HAL → 硬件（如蓝牙校准命令、RTC 设置命令）。这种清晰的数据与控制流向是系统可维护性的基础。

2.3 核心硬件选型与关键参数

主控芯片 STM32F103ZE。 该芯片基于 ARM Cortex-M3 内核，主频最高 72 MHz，内置 512 KB Flash 与 64 KB SRAM，集成 2 路 I²C、3 路 USART、多路 GPIO 与丰富的定时器资源，能够同时驱动 PPG 传感器、加速度计、OLED 显示屏、蓝牙模块与语音模块。其高容量 Flash 与 SRAM 为 FreeRTOS 内核、任务栈、堆内存与算法静态缓冲区提供了充足空间。本项目使用 Keil MDK 5 + ARMCLANG V6.24（AC6）

编译器进行开发，工程文件为 TASK1.uvprojx。

MAX30102 心率血氧传感器。 MAX30102 是 Maxim Integrated 推出的高灵敏度脉搏血氧仪与心率传感器，内部集成红光 LED（660 nm）、红外 LED（880 nm）、光电二极管、低噪声模拟前端与 18 位 ADC。在 SpO2 模式下，可同时采集红光与红外两路 PPG 信号，支持 50~1600 Hz 采样率与 69~411 μ s 脉冲宽度配置。本项目配置为 100 Hz 采样率、411 μ s 脉冲宽度、ADC 量程 4096、LED 电流约 6.4 mA（寄存器值 0x50），兼顾信号质量与功耗。其 FIFO 深度为 32 样本，按 100 Hz 计算约 320 ms 即会满溢，因此需要以足够高的频率读取。

MPU6050 六轴运动传感器。 MPU6050 集成三轴加速度计与三轴陀螺仪，并内置温度传感器。本项目主要使用其加速度计进行计步，同时利用片内温度传感器提供环境温度参考值用于体温补偿。加速度计量程配置为 $\pm 2g$ ，对应灵敏度 16384 LSB/g；片内温度传感器的转换公式为 $T = 36.53 + raw/340\text{ }^{\circ}\text{C}$ 。

DS18B20 数字温度传感器。 DS18B20 采用单总线协议，支持 9~12 位可编程分辨率，12 位模式下转换时间约 750 ms，精度 $\pm 0.5\text{ }^{\circ}\text{C}$ 。本项目通过 PA1 引脚进行 Bit-banging（软件模拟时序）操作，采用“启动转换 $\rightarrow$ 等待 $\geq 750\text{ ms}$ $\rightarrow$ 读取 scratchpad”的两步异步模式，与 MAX30102 的周期性读取并行执行，避免阻塞传感器任务。

DS1302 实时时钟。 DS1302 是低功耗涓流充电时钟芯片，通过 CE、DAT、CLK 三线与主控通信，内置 32.768 kHz 晶振，支持秒、分、时、日、月、年计时，并可通过 CR2032 纽扣电池在主电源断开后维持走时。

SSD1306 OLED 显示屏。 0.96 英寸 128 $\times$ 64 点阵 OLED，通过 I²C 接口通信，内置显存与电荷泵，支持水平寻址模式。本项目通过 I²C2 独立总线连接，并在软件层实现 1024 字节（128 $\times$ 8 page）framebuffer 与脏页标记，仅刷新内容发生变化的页，降低总线负载。

HC-05 蓝牙模块。 经典蓝牙 2.0 SPP 透传模块，默认波特率 9600，支持 AT 指令配置。本项目通过 USART1 与其通信，上电时进行波特率自动探测与回退，正常运行期间默认每 10 秒发送一条健康数据报文（可配置为 2 秒）。

ASR PRO 离线语音识别模块。 该模块在本地完成语音指令识别，通过 USART2 以单字节命令码形式输出识别结果，无需云端连接，响应快速且隐私性高。本项目烧录了 6 条指令，分别对应查看心率、查看步数、归零步数、查看血氧、查看体温、查看状态。

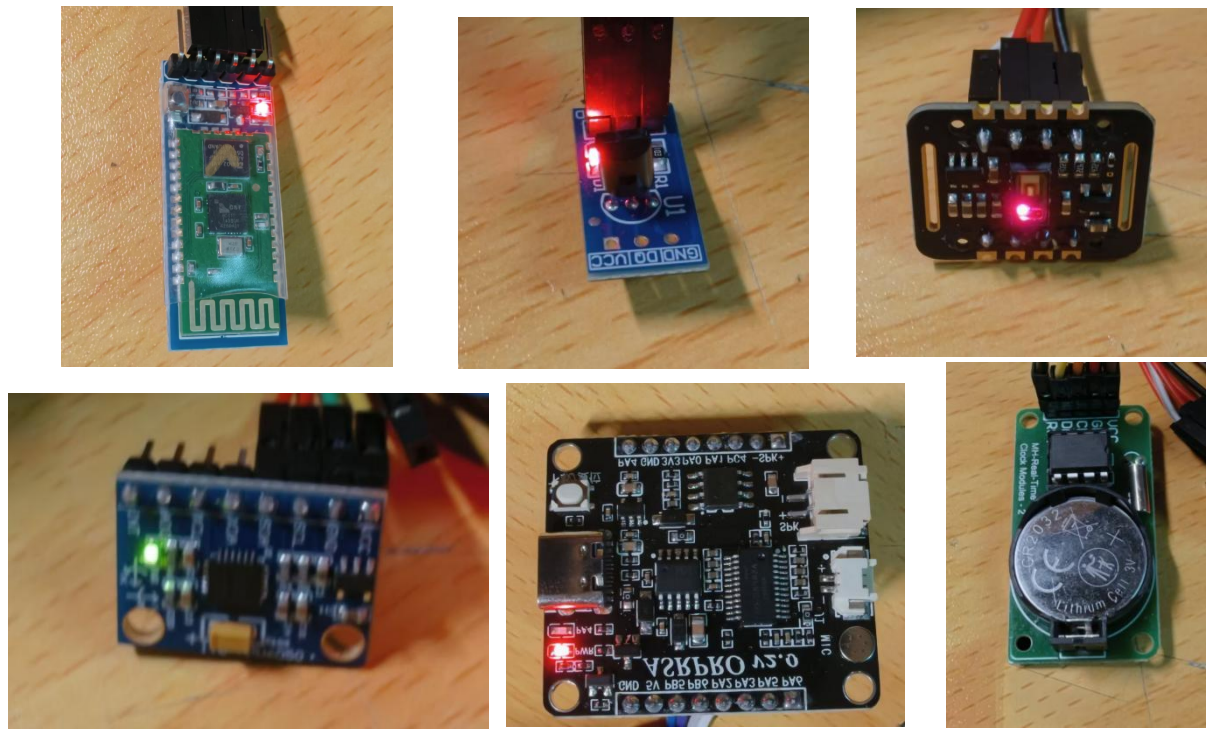


图 2-3 模块实物图

2.4 系统引脚分配与接口规划

引脚规划是硬件设计的基础，也是后续软件驱动开发的前提。STM32F103ZE 拥有 144 个引脚，资源丰富，本系统仅使用其中一小部分 GPIO 与外设引脚，如表 2-2 所示。所有 I²C 信号线均配置为复用开漏（AF_OD）模式，并外接 4.7 k Ω 上拉电阻到 3.3 V；USART 引脚配置为复用推挽输出（TX）与上拉输入（RX）；1-Wire 与三线 RTC 引脚通过软件 Bit-banging（软件模拟时序）实现时序。

表 2-2 系统引脚分配表

引脚	功能	外设/模式	关联文件	说明
PA0	按键 WK_UP	GPIO Out OD	main.c	开漏输出低电平，按键按下被 VCC 拉高
PA1	DS18B20 DQ	1-Wire（GPIO）	ds18b20.h	单总线，需 4.7 k Ω 上拉
PA2	ASR PRO TX	USART2 TX	uart.c	复用推挽输出
PA3	ASR PRO RX	USART2 RX	uart.c	上拉输入，防悬空噪声
PA9	HC-05 TX	USART1 TX	uart.c	复用推挽输出
PA10	HC-05 RX	USART1 RX	uart.c	上拉输入
PB1	DS1302 CLK	GPIO Bit-banging （软件模拟时序）	ds1302.h	推挽输出
PB8	I2C1 SCL	I2C1 Remap	i2c.c	复用开漏，4.7 k Ω 上拉
PB9	I2C1 SDA	I2C1 Remap	i2c.c	复用开漏，4.7 k Ω 上拉

引脚	功能	外设/模式	关联文件	说明
PB10	I2C2 SCL	I2C2	i2c.c	复用开漏，4.7 kΩ 上拉
PB11	I2C2 SDA	I2C2	i2c.c	复用开漏，4.7 kΩ 上拉
PB13	DS1302 CE	GPIO Bit-banging (软件模拟时序)	ds1302.h	推挽输出
PB14	DS1302 DAT	GPIO Bit-banging (软件模拟时序)	ds1302.h	双向，推挽输出/上拉输入

I²C 引脚重映射的必要性。 STM32F103ZE 的 I2C1 默认引脚为 PB6/PB7，但本系统选择将其重映射到 PB8/PB9。原因有二：一是 PB6/PB7 在某些最小系统板上可能已被其他功能占用或布线不便；二是 PB8/PB9 与 I2C2 的 PB10/PB11 在物理位置上相邻，便于走线与管理。重映射通过 `GPIO_PinRemapConfig(GPIO_Remap_I2C1, ENABLE)` 实现，只需在初始化时设置一次。

上拉电阻的选择。 I²C 与 1-Wire 总线都需要外部上拉电阻。上拉电阻过小会增加总线功耗并可能超过器件灌电流能力；过大则会导致上升沿变慢，影响高速通信。本系统选择 4.7 kΩ，这是 100 kbps I²C 与短距离 1-Wire 的常用值。在 I²C 总线较长或从机较多时，可适当减小到 3.3 kΩ 甚至 2.2 kΩ 以改善上升沿；在 1-Wire 总线较长时，也可适当调整。

按键引脚的合理使用。 PA0 是 STM32 的 WK_UP 唤醒引脚，内部通常不带上拉电阻。本系统将其配置为开漏输出并主动输出低电平，外部按键一端接地、另一端接 VCC。当按键未按下时，开漏输出保持低电平，PA0 读到低电平；当按键按下时，VCC 通过按键将 PA0 拉高。这种设计无需外部上拉电阻，且消抖逻辑简单。

2.5 电源与时钟设计方案

电源与时钟是嵌入式系统的两大基础，直接影响系统稳定性、测量精度与功耗。

电源设计。 系统采用 5 V USB 输入作为总电源，经过 AMS1117-3.3 线性稳压器降压至 3.3 V，为 STM32F103ZE、MAX30102、MPU6050、DS18B20、DS1302、SSD1306、HC-05 与 ASR PRO 供电。AMS1117 的最大输出电流为 1 A，压差约为 1 V，在 5 V 输入、3.3 V 输出、100 mA 负载时功耗约为 $(5-3.3)V \times 0.1A = 0.17\text{ W}$ ，无需额外散热。在系统整体功耗接近 70 mA 时，稳压器功耗约为 0.12 W，属安全范围。

在电源完整性方面，建议在 AMS1117 输入输出端分别放置 10 μF 钽电容或陶瓷电容，用于抑制低频纹波；在每个传感器模块的 VCC 与 GND 之间放置 100 nF 陶瓷电

容，用于滤除高频噪声。良好的去耦设计能够减少数字电路切换对模拟传感器（尤其是 MAX30102 光电二极管前端）的干扰，提高 PPG 信号质量。

时钟设计。 STM32F103ZE 内部集成 8 MHz HSI 与 40 kHz LSI 两个 RC 振荡器，外部可接 8 MHz HSE 晶振与 32.768 kHz LSE 晶振。本系统使用外部 8 MHz HSE 作为系统时钟源，通过 PLL 倍频至 72 MHz，为 CPU 与大部分外设提供时钟。72 MHz 是 STM32F103ZE 的最高主频，能够确保 FreeRTOS 1 ms 节拍的精度、DWT 延时的微秒级分辨率以及 PPG 算法的实时性。

LSI 用于 IWDG 独立看门狗，其频率约为 40 kHz，经 256 分频与 625 重载后得到 4 s 超时。由于 LSI 精度较低（典型 $\pm 40\%$ ），实际超时时间可能在 2.4~5.6 s 之间变化，但对于“检测死锁并复位”这一应用而言，如此大的容差完全可以接受。若需要更精确的看门狗超时，可使用外部 LSE 晶振驱动窗口看门狗（WWDG），但本系统未使用 WWDG。

2.6 通信协议选型对比

本系统同时采用 I²C、UART、1-Wire 与 GPIO Bit-banging（软件模拟时序）四种通信方式，各协议的选择依据如下。

I²C： 适用于多寄存器配置、多从机共享总线的传感器。MAX30102、MPU6050、SSD1306 均支持 I²C，因此统一使用该协议可以减少 GPIO 占用。I²C 的缺点是半双工、需要上拉电阻、对总线电容敏感、时序相对复杂。本系统通过双 I²C 总线与错误恢复机制克服了其带宽与可靠性问题。

UART（Universal Asynchronous Receiver/Transmitter）： 适用于点对点异步串行通信。HC-05 与 ASR PRO 均使用 UART，因为其协议简单、无需额外时钟线、易于与 PC 或模块对接。UART 的缺点是波特率必须双方一致，且长距离传输易受干扰。本系统通过波特率自动探测与 RX 上拉输入提高了 UART 的易用性与可靠性。

1-Wire： DS18B20 专属协议，只需要一根数据线即可完成供电与通信。其优点是布线极简，缺点是时序严格、对微秒级延时要求高、总线电容限制传输距离。本系统通过 DWT 精确延时与临界区保护实现了可靠的 1-Wire 通信。

2.7 软件开发环境与工具链

本项目采用 Keil MDK（uVision）5 作为集成开发环境，配合 ARMCLANG V6.21（AC6）编译器。Keil MDK 是 STM32 开发的主流工具之一，提供代码编辑、编译、

链接、调试、下载一站式支持。工程文件 `TASK1.uvprojx` 中定义了目标器件、编译选项、包含路径、源文件分组与调试配置。

编译器配置。 工程启用 C99 标准（`uC99=1`）与 GNU 扩展（`uGnu=1`），以支持部分 GCC 风格语法。优化等级设为 2（`-O2`），在代码大小与执行速度之间取得平衡。预定义宏包括 `USE_STDPERIPH_DRIVER`（启用 STM32 标准外设库）与 `STM32F10X_HD`（告诉编译器使用大容量器件头文件）。包含路径覆盖了标准外设库头文件、CMSIS 核心头文件、FreeRTOS 头文件、`crouse.c` 与 `crouse.h` 目录。

调试工具。 开发阶段使用 ST-Link V2 或兼容调试器通过 SWD 接口连接 STM32F103ZE，实现单步调试、断点、变量观察、内存查看与程序下载。Keil 的调试视图可以观察 GPIO、USART、PC 等外设寄存器状态，便于定位通信故障。

版本控制。 项目使用 Git 进行版本管理，`crouse.c` 与 `crouse.h` 为用户代码目录，`Device`、`Core`、`freertos` 为第三方或库代码目录。通过 `.gitignore` 排除了编译生成的 `Objects`、`Listings` 与 Keil 用户配置文件，保持仓库整洁。

辅助工具。 串口调试助手用于观察 USART1 输出；逻辑分析仪用于抓取 I²C、1-Wire、DS1302 波形；Excel 或 Python 脚本用于分析 PPG 原始数据与算法输出；字模生成工具用于制作 `font.h` 中的 12×12 中文字体。

3 软件设计与实现

3.1 四层软件架构设计

本系统软件采用四层架构，自上而下依次为应用层、算法层、驱动层与硬件抽象层（HAL）。每一层仅与相邻下层交互，层间通过定义明确的函数接口通信，避免跨层调用与循环依赖。该分层设计有多重优势：一是模块职责清晰，便于定位问题；二是上层逻辑不依赖具体硬件型号，便于更换传感器或主控平台；三是教学与维护人员可从任意一层切入阅读代码，无需通读全部实现。

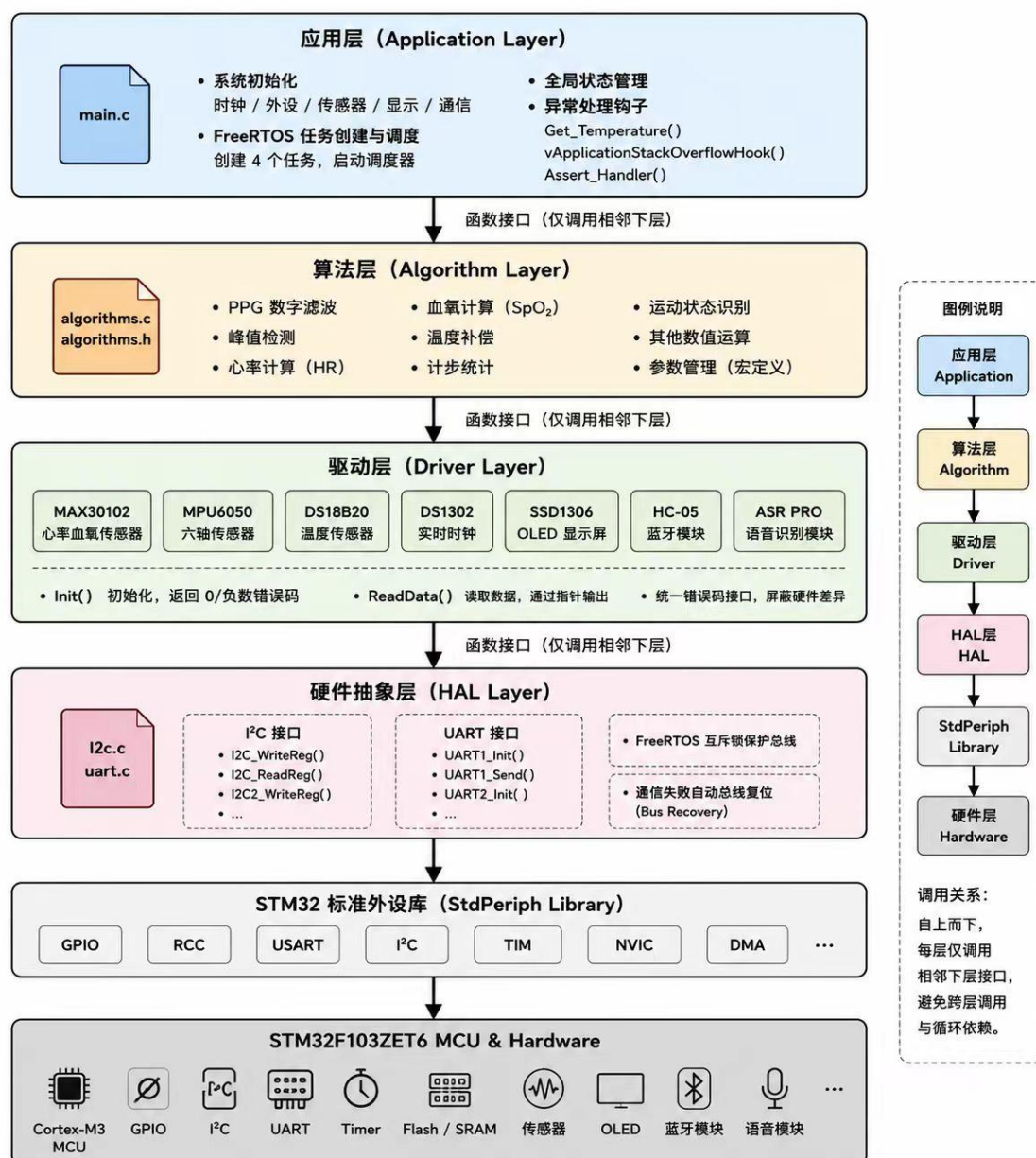
应用层对应 `main.c`，负责系统初始化、FreeRTOS 任务创建、全局状态管理与异常处理钩子。`main()` 函数按顺序完成时钟、外设、传感器等模块的初始化，创建四个 FreeRTOS 任务后启动调度器，同时还实现了 `Get_Temperature()` 等支撑函数。

算法层对应 `algorithms.c` 与 `algorithms.h`，是系统的计算中枢，接收驱动层原始数据，完成 PPG 滤波、数值计算等全部运算。算法层不访问硬件寄存器，输入通过函数参数传入，可调参数以宏形式集中在 `algorithms.h` 中，便于教学演示与参数调优。

驱动层包含多个硬件对应文件，每个驱动封装对应硬件的初始化与数据读取函数，统一错误码与数据输出约定，通过调用 HAL 层函数与硬件交互，不直接操作寄存器时序细节。

硬件抽象层对应 i2c.c 与 uart.c，负责封装 STM32 标准外设库的寄存器操作，生成统一可重入接口：i2c.c 提供寄存器读写函数，内部用互斥锁保护总线，通信失败时自动复位；uart.c 提供初始化、发送函数，配置通信相关参数。

图 3-1 四层软件架构



四层架构的调用关系可概括为：应用层调用算法层与驱动层；算法层调用驱动层获取数据；驱动层调用 HAL 层完成通信；HAL 层调用 STM32 StdPeriph 库或直接访问

寄存器；最底层为 MCU 硬件与外设。

3.2 FreeRTOS 多任务调度设计

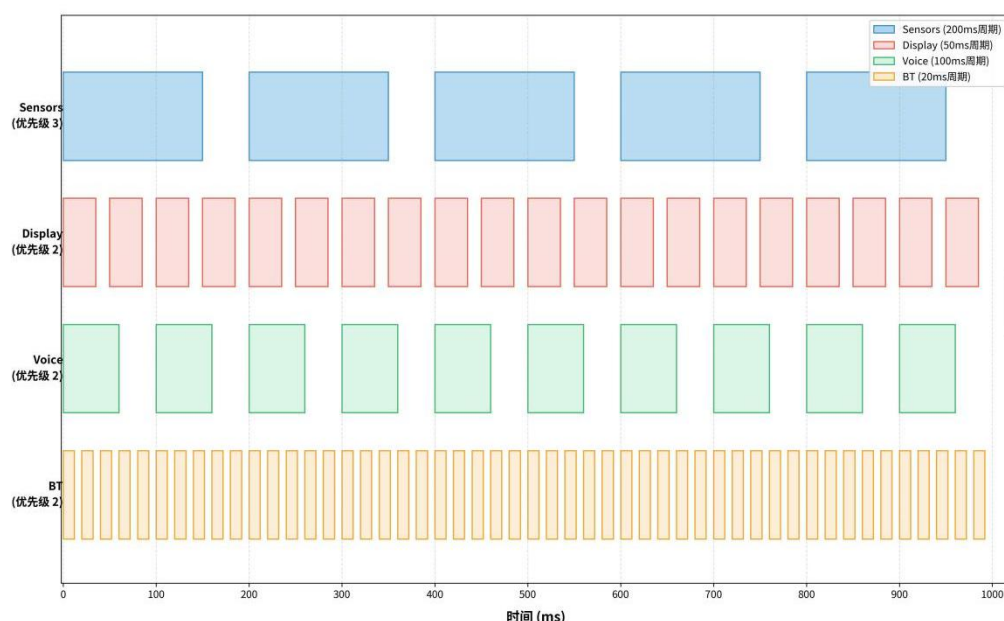


图 3-2 FreeRTOS 任务调度

3.2.1 实时操作系统选型与配置

本系统选用 FreeRTOS 基于以下考量：内核极小（约 9 KB Flash、约 1 KB RAM），在 STM32F103ZE 资源约束下占用最少；抢占式调度与互斥锁等机制能自然映射到本系统的四类并发业务；与 Keil MDK 集成成熟，社区文档丰富。关键配置参数如下。configTICK_RATE_HZ 设为 1000（1 ms 节拍），这是在实时性与开销之间的折中：1 ms 节拍足以满足 200 ms 传感器任务与 50 ms 显示任务的精度要求，同时避免了更高频率（如 10 kHz）导致的上下文切换开销；configMAX_PRIORITIES 设为 5（0~4 优先级），本系统实际使用 3 个优先级（Sensors=3、Display/Voice/BT=2、Idle=0），预留 2 个优先级供未来扩展；configTOTAL_HEAP_SIZE 设为 15 KB，经实测四个任务栈（约 7 KB）加上 TCB、互斥锁等内核对象后剩余约 3~5 KB 余量，既避免了内存浪费，又为未来功能扩展预留了空间；configUSE_TIMERS 设为 0 禁用软件定时器，因为本系统的定时需求（如蓝牙 10 秒上报）通过任务内轮询即可实现，禁用后可节省约 1.2 KB 堆空间。。

FreeRTOS 的配置集中在 `FreeRTOSConfig.h` 中。关键配置项如下：
`configUSE_PREEMPTION` 设为 1，启用抢占式调度；`configTICK_RATE_HZ` 设为 1000，即 1 ms 调度节拍；`configMAX_PRIORITIES` 设为 5，提供 0~4 共 5 个优先级；

`configTOTAL_HEAP_SIZE` 设为 15 KB，用于任务栈、TCB、互斥锁与队列的动态分配；`configCHECK_FOR_STACK_OVERFLOW` 设为 1，启用方法 1 栈溢出检测；`configUSE_MUTEXES` 与 `configUSE_RECURSIVE_MUTEXES` 均设为 1，支持互斥锁；`configUSE_TIMERS` 设为 0，禁用软件定时器以节省约 1.2 KB 堆空间；`configASSERT` 宏映射到 `Assert_Handler()`，用于捕获内核断言失败。

`configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY` 设为 5，确保高优先级中断（数值 ≤ 5 ）不调用 FreeRTOS 系统调用。

FreeRTOS 调度由 SysTick 提供系统节拍、PendSV 执行任务切换，中断处理不会被任务切换打断，兼顾了实时性与响应效率。

Cortex-M3 的硬件支持使得 FreeRTOS 上下文切换高效，典型切换时间仅为数十个时钟周期。

`configIDLE_SHOULD_YIELD` 设为 1，表示当 Idle 任务与优先级 0 的用户任务共享 CPU 时，Idle 任务会主动让出时间片。本系统未使用优先级 0 的用户任务，因此该配置主要影响 Idle 任务的行为。`configUSE_TRACE_FACILITY` 设为 0，关闭了任务追踪功能以节省代码空间；若需要进行调度分析，可将其设为 1 并通过 `vTaskList()` 获取任务状态。

3.2.2 任务划分、优先级与周期设计

系统共创建四个用户任务，其优先级、周期、栈大小与职责如表 3-1 所示。所有任务均在 `main()` 中通过 `xTaskCreate()` 创建，第三个参数为栈大小（单位：字，乘以 4 即为字节数）。

表 3-1 FreeRTOS 任务参数表

任务名	栈大小（字）	优先级	周期	主要职责
Sensors	512	3	200 ms	读取 MAX30102 FIFO、MPU6050 加速度与温度、DS18B20 温度转换
Display	512	2	50 ms	按键消抖、页面切换、OLED 刷新
Voice	384	2	100 ms	轮询 UART2 接收 ASR PRO 单字节命令
BT	384	2	20 ms	轮询 UART1 接收蓝牙指令、定时上报健康数据

Sensors 任务优先级最高（3），原因有两点：一是 MAX30102 FIFO 深度仅 32 样本，100 Hz 采样率下约 320 ms 即会满溢，若 Sensors 任务被长时间阻塞将导致样本丢失；二是心率和血氧算法需要连续、完整的数据流才能稳定收敛。Display、Voice、BT 任务优先级均为 2，它们之间通过时间片轮转共享 CPU。BT 任务周期最短（20

ms），是为了及时处理蓝牙接收缓冲区中的 AT 指令或校准命令；Voice 任务 100 ms 周期足以覆盖 ASR PRO 的识别响应时间；Display 任务 50 ms 周期用于按键扫描与 500 ms 一次的页面刷新。

任务间数据交换采用共享全局变量模式：`hr`、`spo2`、`steps`、`activity_state` 定义在 `algorithms.c` 中并声明为 `volatile int`；`g_temperature` 与 `g_temp_valid` 定义在 `main.c` 中并声明为 `volatile`；`g_page_advance` 在 `globals.h` 中声明。Cortex-M3 对 32 位对齐的 `int` 读取是原子的，因此读取这些变量时不会出现“撕裂值”，但读者可能看到比写入时刻稍旧的数据，这对于显示与蓝牙上报场景是可接受的。体温变量是 `float` 类型，且需要同时读取有效标志，因此专门提供 `Get_Temperature()` 函数，在临界区内原子快照 `g_temp_valid` 与 `g_temperature`，避免读到“有效标志为 1 但温度值仍是旧值”的 `inconsistent` 状态。

3.2.3 栈溢出检测与断言诊断

`configCHECK_FOR_STACK_OVERFLOW` 设置为 1 后，FreeRTOS 会在任务切换时检查栈底附近的“哨兵值”是否被改写。若检测到栈溢出，内核调用 `vApplicationStackOverflowHook()`。本实现中，该钩子首先将 UART1 重新初始化为 115200 bps（调试波特率），然后发送字符串“STACK OVERFLOW: <taskname>\r\n”，最后进入死循环。由于 IWDG 看门狗在循环顶部未被喂狗，系统将在 4 s 内自动复位，避免故障持续扩散。

`configASSERT(x)` 被定义为：当条件 `x` 为 0 时，调用 `Assert_Handler(__FILE__, __LINE__)`。`Assert_Handler()` 同样重新初始化 UART1 到 115200 bps，发送“ASSERT FAIL: <file>:<line>\r\n”，然后死循环等待看门狗复位。该机制对于定位互斥锁创建失败、参数越界、内核内部错误等问题非常有价值。需要注意的是，`Assert_Handler()` 可能在两种时机触发：一是调度器启动前，此时 UART1 尚未初始化，因此钩子必须主动调用 `UART1_Init(115200)`；二是运行期 UART1 已被 `HC05_Init()` 切到 9600 bps，钩子再次将其切回 115200 仅用于输出诊断信息，随后即死循环，不影响正常运行逻辑。

3.2.4 调度时序与任务协作关系

系统启动后，Sensors 任务每 200 ms 执行一次循环。在每次循环中，它依次调用 `MAX30102_ReadData()` 读取 PPG FIFO、调用 `MPU6050_ReadData()` 读取加速度与芯片温度、根据 `ds18b20_tick` 计数器状态启动或读取 DS18B20 温度转换。当 `ds18b20_tick` 达到 5

时，读取温度后将计数器清零并通过 `continue` 跳过 `vTaskDelay(200)`，立即开始下一轮。这一设计使得 DS18B20 的 800 ms 转换周期与 MAX30102 的 200 ms 读取周期实现周期同步：在第 0 个 tick 启动转换，在第 1~4 个 tick 等待，在第 5 个 tick 读取，然后无缝进入下一个 6-tick 周期。

Display 任务每 50 ms 检查一次按键状态。按键采用无阻塞消抖：由于调用周期 50 ms 远大于机械抖动时间（<10 ms），单次毛刺不会构成“持续按下”；同时记录上一次按键时间，确保两次有效触发间隔不小于 300 ms，防止连按。当检测到按键或语音命令触发翻页时，将 `tick` 置为 `DISPLAY_FORCE_REFRESH`（16），强制在下一轮立即刷新页面，而无需等待 500 ms 周期。

Voice 任务每 100 ms 调用一次 `ASR_ProcessUART()`。该函数首先检查并清除 UART2 的 Overrun Error（ORE）标志，然后读取 RXNE 接收缓冲区中的所有字节。每个字节作为一个命令码，通过 `switch-case` 映射到对应的 `OLED_Show*` 函数。ASR PRO 模块在识别到语音后会主动发送单字节，STM32 只需轮询接收即可。

BT 任务每 20 ms 调用一次 `HC05_Process()`。该函数轮询接收缓冲区以处理可能的 AT 指令或校准命令（虽然当前版本主要使用自动波特率探测，运行时命令解析已简化），并通过 `last_report_tick` 计数器每 10 秒（代码中 `HC05_REPORT_INTERVAL_MS` 为 10000，即 10 秒；若需要 2 秒可修改该宏）发送一次健康数据报文。报文包含心率、血氧、步数与体温，通过 `Get_Temperature()` 原子读取体温。

3.2.5 优先级反转、互斥锁继承与临界区策略

在多任务实时系统中，优先级反转（Priority Inversion）是一个经典且危险的并发问题。其典型场景为：低优先级任务持有共享资源（如互斥锁），中等优先级任务抢占了低优先级任务，而高优先级任务因等待该共享资源被阻塞。结果，高优先级任务被间接地阻塞了中等优先级任务的执行时间，违反了优先级调度的初衷。FreeRTOS 的互斥锁（Mutex）通过优先级继承机制来缓解这一问题：当高优先级任务因等待互斥锁而阻塞时，持有互斥锁的低优先级任务会临时提升到高优先级任务的优先级，从而尽快释放资源；释放后，低优先级任务恢复原始优先级。

本系统中，PC 互斥锁 `g_i2c_mutex` 与 `g_i2c2_mutex` 均使用 FreeRTOS 的 `xSemaphoreCreateMutex()` 创建，因此天然支持优先级继承。例如，当 Sensors 任务（优先级 3）尝试获取 `g_i2c_mutex` 时，若 Display 任务（优先级 2）正持有该锁，Display 任务将临时继承 Sensors 的优先级 3，直到释放锁。这一机制确保了高优先级的传感器

数据采集不会被低优先级的显示任务长时间阻塞。

临界区（Critical Section）是另一种保护共享资源的机制，通过 `taskENTER_CRITICAL()` 与 `taskEXIT_CRITICAL()` 关闭和恢复可屏蔽中断优先级以上的中断。与互斥锁不同，临界区不依赖任务调度，适用于非常短的原子操作。本系统将临界区用于以下场景：DS18B20 的 1-Wire Bit-banging（软件模拟时序）时序、DS1302 的三线时序、体温全局变量的读取。这些操作要么对时序敏感（不允许任务切换打断微秒级时序），要么非常短暂（仅读取两个变量），使用临界区比互斥锁更高效且安全。

需要注意的是，临界区内不能调用任何可能阻塞的 FreeRTOS API（如 `vTaskDelay()`、`xSemaphoreTake()`），因为调度器依赖中断，关闭中断后阻塞将导致死锁。本系统的临界区使用严格遵守这一原则：所有临界区代码都是纯计算或 GPIO Bit-banging（软件模拟时序），不含阻塞调用。

3.3 硬件抽象层设计：I²C 与 UART

3.3.1 双 I²C 总线架构设计思路

I²C（Inter-Integrated Circuit）是一种同步、半双工、多主从串行总线，只需要两根信号线（SCL 时钟线与 SDA 数据线），通过开漏输出与外接上拉电阻实现线与逻辑。STM32F103ZE 内置 I2C1 与 I2C2 两个硬件 I²C 接口，其中 I2C1 的默认引脚为 PB6/PB7，可通过 AFIO 重映射到 PB8/PB9。

本系统的一个关键设计决策是将 OLED 显示屏单独挂接到 I2C2，而将 MAX30102 与 MPU6050 挂接到 I2C1。该决策并非随意为之，而是基于以下工程考量：MAX30102 在 100 Hz 采样率、每样本 6 字节的配置下，每 200 ms 传感器任务最多需要从 FIFO 读取 32 个样本，即 192 字节；再加上 MPU6050 每 200 ms 读取 8 字节加速度加温度数据；I2C1 的理论带宽为 100 kbps，虽然足以容纳上述数据，但如果 OLED 的 128 字节全屏刷新也共享同一条总线，则可能在显示刷新高峰期延迟 PPG FIFO 读取，导致 FIFO 溢出。将 OLED 移到独立的 I2C2 后，显示任务与传感器任务在 I²C 层面完全解耦，这是保障 PPG 数据连续性的关键设计之一。

I2C1 配置为重映射到 PB8/PB9，原因是为了将 PB6/PB7 保留给其他可能用途，同时 PB8/PB9 在 STM32F103ZE 的 LQFP144 封装上位置更集中，便于布线。I2C1 与 I2C2 均配置为标准模式 100 kbps、占空比 2、7 位寻址、使能应答。

3.3.2 I²C 事务封装与互斥保护

`i2c.c` 中实现了统一的 `i2c_xfer()` 函数，它接受 I2C 外设指针、互斥锁句柄、设备地址、寄存器地址、数据缓冲区、长度与读写方向，完成一次完整的 I²C 寄存器读写事务。函数流程如下：

- 尝试获取互斥锁，超时时间为 100 ms。若获取失败直接返回 -1，避免无限阻塞。
- 发送 START 条件与 7 位设备地址加写方向位，等待 ADDR 标志。
- 发送寄存器地址，等待 TXE 标志。
- 若是读事务，重新发送 START 与设备地址加读方向位，然后循环接收数据；在最后一个字节前关闭 ACK 并发送 STOP，读取最后一个字节。
- 若是写事务，循环发送数据，最后发送 STOP。
- 若任何步骤失败，发送 STOP 并调用 `i2c_bus_reset()` 复位总线，然后释放互斥锁。

`i2c_wait()` 函数用于等待 SR1 状态寄存器中的指定标志位置位。它不仅检查目标标志，还同时监控 AF（从机不应答）、BERR（总线错误）、ARLO（仲裁丢失）三种错误标志。一旦这些错误标志置位，目标标志将永远不会出现，因此函数立即返回 -1，避免傻等满 50 ms 超时。这种“快速失败”策略对于实时系统至关重要。

`i2c_bus_reset()` 函数在发生错误时被调用，执行 `I2C_DeInit()` followed by `I2C_Init()`，重新配置总线参数并启用外设。该机制可以恢复因 SDA 被从机拉低、START/STOP 异常、总线冲突等原因导致的总线挂死状态。

互斥锁 `g_i2c_mutex` 与 `g_i2c2_mutex` 分别在 `I2C1_Init()` 与 `I2C2_Init()` 中创建。由于这两个初始化函数在 `main()` 中于 `vTaskStartScheduler()` 之前调用，因此创建互斥锁是安全的——FreeRTOS 的队列/信号量创建函数不依赖调度器运行。这种设计保障了从系统启动开始，任何 I²C 访问都受互斥锁保护。

3.3.3 UART 初始化与发送封装

`uart.c` 封装了 USART1 与 USART2 的初始化与发送功能。USART1 用于 HC-05 蓝牙通信与调试输出，USART2 用于 ASR PRO 语音识别。两者均配置为 8 数据位、1 停止位、无校验、无硬件流控。

`UART1_Send()` 采用轮询方式发送数据，对每个字节等待 TXE（发送数据寄存器空）标志，全部发送完成后等待 TC（发送完成）标志。发送过程带有 200 ms 超时保

护，防止在蓝牙模块未连接或总线异常时无限阻塞。

USART RX 引脚配置为上拉输入（GPIO_Mode_IPU），这是本系统的一个重要可靠性设计。当 ASR PRO 或 HC-05 模块未上电或未接线时，RX 引脚不会悬空，而是被内部上拉电阻稳定在 3.3 V 附近，避免因浮空输入引入噪声而导致的误触发或误接收。同时，在 HC05_Process() 与 ASR_ProcessUART() 中，每次轮询都会检查并清除 ORE（Overrun Error）标志，防止接收缓冲区溢出后 UART 停止接收新数据。

3.3.4 I²C 通信时序与错误恢复实例分析

为了更具体地理解 I²C 通信中的错误处理，下面以读取 MAX30102 的 PART_ID 寄存器为例，说明一次完整的 I²C 读事务流程及可能的错误点。PART_ID 位于寄存器 0xFF，固定返回 0x15，是验证器件在线的首选寄存器。

一次读事务由 I2C_ReadReg(MAX30102_I2C_ADDR, 0xFF, &val, 1) 触发，内部调用 i2c_xfer(I2C1, g_i2c_mutex, 0xAE, 0xFF, &val, 1, 1)。执行步骤如下：

- **获取互斥锁：**调用 xSemaphoreTake(g_i2c_mutex, 100)。若 100 ms 内未获得锁，返回 -1。这通常发生在 I2C1 被其他任务长时间占用或发生死锁时。
- **发送 START：**I2C_GenerateSTART(I2C1, ENABLE)，然后等待 I2C_SR1_SB（起始位已发送）标志。若 SCL 或 SDA 被外部设备拉低，START 条件无法生成，i2c_wait() 监控到 AF/BERR/ARLO 错误或超时后返回 -1。
- **发送设备地址（写）：**I2C_Send7bitAddress(I2C1, 0xAE, Transmitter)，等待 I2C_SR1_ADDR 标志。若 MAX30102 未连接或地址错误，从机不会 ACK，AF 标志置位，i2c_wait() 返回 -1。
- **发送寄存器地址：**I2C_SendData(I2C1, 0xFF)，等待 I2C_SR1_TXE。若 TXE 长时间不置位（例如总线挂死），超时返回 -1。
- **重复 START 并发送设备地址（读）：**等待 I2C_SR1_ADDR 后读取 SR2 清零 ADDR 标志。
- **接收数据：**由于只读 1 字节，提前关闭 ACK 并发送 STOP，然后等待 RXNE，读取数据。
- **释放互斥锁：**无论前面是否出错，函数最后都会执行 xSemaphoreGive(g_i2c_mutex)。

若任何步骤返回 -1，i2c_xfer() 会执行 I2C_GenerateSTOP() 与 i2c_bus_reset()。i2c_bus_reset() 调用 I2C_DeInit() 关闭外设，重新填充 I2C_InitTypeDef 结构体，调用

`I2C_Init()` 与 `I2C_Cmd(ENABLE)` 重启外设。这一过程相当于对 I²C 外设进行一次“软复位”，能够恢复绝大多数总线异常状态。

3.3.5 UART 波特率自动探测原理

HC-05 蓝牙模块出厂时可能处于不同波特率（常见 9600 或 38400），若 STM32 以错误波特率发送 AT 指令，模块将无法识别。`HC05_Init()` 实现了波特率自动探测与回退机制，其核心逻辑如下：

- 定义尝试列表：`{baud, 38400, 115200, 57600, 19200}`，首选目标波特率，其余为常见可能波特率。
- 对每个波特率，调用 `UART1_Init(baud)` 初始化串口，发送 "AT\r\n"，然后在 500 ms 内轮询接收缓冲区。
- 若接收到包含 "OK" 的响应，说明当前波特率正确。若该波特率不是目标波特率，则发送 `AT+UART=<baud>,0,0\r\n` 命令修改模块波特率，等待 200 ms 后重新以目标波特率初始化串口。
- 若所有波特率都未收到 "OK"，则放弃探测，直接以目标波特率初始化串口，寄希望于模块已在目标波特率但可能未响应 AT 指令。

这种自动探测机制提高了系统的兼容性，用户无需手动设置 HC-05 波特率即可使用。需要注意的是，自动探测期间会短暂改变 UART1 波特率，因此不应在探测过程中发送重要数据。

3.4 传感器驱动层设计

3.4.1 统一传感器 API 设计模式

本系统所有传感器驱动遵循统一的 API 设计模式：每个传感器由一个 `.c` 文件实现，一个 `.h` 文件声明公共接口与可配置参数。`.h` 文件中集中定义 I²C 地址、寄存器地址、引脚分配、量程、电流、校准常数等所有可调参数；`.c` 文件中实现初始化与数据读取逻辑。上层代码只需阅读 `.h` 文件即可了解如何调用该传感器，无需深入 `.c` 的实现细节。

函数命名约定如下：`Init()` 函数返回 `int`，0 表示成功，负数表示错误码；`ReadData()` 函数返回 `int`，0 表示成功，负数表示错误，数据通过指针参数输出；对于需要转换延迟的异步传感器（如 DS18B20），额外提供 `StartXxx()` 函数启动转换。这种一致性使得 `main()` 中的初始化序列与 `vTaskSensors()` 中的读取序列非常规整，便于

阅读与维护。

3.4.2 MAX30102 驱动与 FIFO 读取策略

MAX30102 心率血氧传感器。 MAX30102 是 Maxim Integrated 推出的高灵敏度脉搏血氧仪与心率传感器，内部集成红光 LED（660 nm）、红外 LED（880 nm）、光电二极管、低噪声模拟前端与 18 位 ADC。本项目配置为 100 Hz 采样率，LED 电流约 6.4 mA（寄存器值 0x50），兼顾信号质量与功耗。关键参数详见表 2-3。

`MAX30102_ReadData()` 是本驱动的核心。它首先分别读取 FIFO 写指针（`REG_FIFO_WR_PTR, 0x04`）与读指针（`REG_FIFO_RD_PTR, 0x06`），计算可用样本数为 $(wr - rd) \& 0x1F$ 。使用按位与 0x1F 是因为 FIFO 深度为 32，指针以 5 位循环。然后一次性 burst 读取 FIFO 数据寄存器（`REG_FIFO_DATA, 0x07`）的 `samples × 6` 字节，每 6 字节包含红光 18 位与红外 18 位数据。解码时，将三个字节按大端序组合为 32 位整数，再与 0x3FFFF 进行按位与，得到 18 位有效值。最后调用 `PPG_ProcessSamples(ir_dec, red_dec, samples)` 将数据送入算法层。

两个指针分开读取并非原子操作，但这一设计是安全的：我们以写指针 `wr` 的快照为准，计算从当前读指针到 `wr` 之间可用的样本数；如果在两次读取之间有新数据到达，读指针尚未更新，新数据会留到下一次读取；如果在读取 FIFO 数据期间又有新数据写入，FIFO 滚动覆盖不会影响我们已经 burst 读取的数据。因此，这种非原子读取策略在工程上是可接受的。

3.4.3 MPU6050 驱动与批量读取优化

`mpu6050.c` 实现了 MPU6050 的初始化与批量读取。初始化流程包括：向 `PWR_MGMT_1 (0x6B)` 写 0x00 唤醒芯片；向 `ACCEL_CONFIG (0x1C)` 写 `MPU6050_ACCEL_RANGE (0x00, ±2g)`。

`MPU6050_ReadData()` 从 `ACCEL_XOUT_H (0x3B)` 开始连续读取 8 字节，依次解析为 X、Y、Z 三轴加速度（各 2 字节）和片内温度（2 字节）。这种批量读取相比分四次读取单个寄存器，显著减少了 I²C 总线事务数量与开销。温度转换公式为 `chip_temp = 36.53 + raw / 340`，该公式来自 MPU6050 数据手册。当温度指针非空时写入计算结果，否则跳过。

3.4.4 DS18B20 驱动与两步异步模式

DS18B20 采用 Dallas 1-Wire 协议，时序严格，对微秒级延迟敏感。`ds18b20.c` 通过 GPIO Bit-banging（软件模拟时序）实现该协议，主要函数包括 `ow_reset()`、`ow_write_byte()`、`ow_read_byte()`。复位时序为：主机拉低 DQ 500 μs ，释放后等待 60 μs ，再采样 presence 脉冲，最后等待 420 μs 完成时序。写“1”时隙为拉低 5 μs 后释放 65 μs ；写“0”时隙为拉低 65 μs 后释放 5 μs 。读时隙为拉低 2 μs 后释放，等待 8 μs 采样，再等待 55 μs 完成。

DS18B20 在 12 位分辨率下的温度转换时间约为 750 ms，若采用阻塞等待方式，将严重拖慢 Sensors 任务。因此本系统采用两步异步模式：`DS18B20_StartConversion()` 发送 Skip ROM (0xCC) 与 Convert T (0x44) 命令后立刻返回；`vTaskSensors()` 在接下来的 4 个 200 ms 周期内不访问 DS18B20；第 5 个周期调用 `DS18B20_ReadData()` 读取 scratchpad 中的温度值。这种设计将 750 ms 的转换等待时间自然嵌入到 MAX30102 的周期性读取中，无需额外的阻塞延迟。

读取 scratchpad 时，本实现目前读取前两个字节（温度低字节与高字节）即完成计算，注释中提到的 CRC-8 校验在代码中尚未完整实现，后续可扩展为对 9 字节 scratchpad 进行 CRC 校验以提升可靠性。

3.4.5 DS1302 驱动与上电自检

`ds1302.c` 通过 PB1 (CLK)、PB13 (CE)、PB14 (DAT) 三线模拟 SPI-like 时序与 DS1302 通信。写操作时，DAT 配置为推挽输出，每个位在 CLK 低电平期间设置，然后 CLK 拉高；读操作时，DAT 切换为上拉输入，在 CLK 高电平期间采样。`DS1302_Init()` 在初始化时执行一次上电自检：发送读秒命令 (0x81)，若读取值为 0xFF 则判定通信失败；若秒寄存器的 CH 位（时钟暂停位）为 1，则通过写保护寄存器 (0x8E) 关闭写保护，再向秒寄存器 (0x80) 写 0x00 清除 CH 位以启动晶振，最后重新开启写保护。

`DS1302_ReadTime()` 发送突发读命令 (0xBF)，连续读取秒、分、时、日、月、星期、年 7 个 BCD 编码寄存器，忽略星期字段，将其余字段转换为十进制后存入 `rtc_time_t` 结构体。整个读操作在临界区内完成，避免被 FreeRTOS 任务切换打断导致时序错误。

3.4.6 SSD1306 OLED 驱动与脏页刷新

`oled_ssd1306.c` 实现了 SSD1306 的初始化、framebuffer 管理与多种页面显示函

数。初始化命令序列包括：关显示、设置水平寻址模式、设置页起始地址、设置 COM 扫描方向、设置列起始地址、设置显示起始行、设置对比度、设置段重映射、设置正常/反色、设置复用比、开显示、开电荷泵等。

显存 framebuffer 定义为 `static uint8_t framebuf[8][128]`，对应 128×64 分辨率按 8 个 page 组织，每个 page 128 字节。`dirty[8]` 数组标记哪些 page 需要刷新。`OLED_Flush()` 只将 dirty page 的数据通过 `I2C2_WriteReg()` 写入 SSD1306 的显存，每次写入 128 字节数据。这种脏页刷新机制避免了每次全屏刷新带来的 1024 字节 I²C 流量，尤其在部分页面内容未变化时效果显著。

为了防止 `vTaskDisplay` 与 `vTaskVoice` 同优先级时间片轮转导致 Clear→Draw→Flush 序列被中断而产生花屏，`oled_ssd1306.c` 为每个 `OLED_Show*` 上层函数增加了 `g_oled_mutex` 互斥锁。`oled_lock()` 在函数入口获取锁，`oled_unlock()` 在函数出口释放锁。低层函数 `OLED_Clear()`、`OLED_DrawChar()`、`OLED_DrawString()`、`OLED_Flush()` 不单独加锁，由上层 `Show*` 函数保障序列完整性，避免递归加锁或死锁。

3.4.7 字体渲染与页面布局设计

OLED 显示不仅需要驱动芯片能够正确接收数据，还需要在软件层实现字体渲染、页面布局与中文显示。

英文字体。 系统使用 5×7 点阵字体，每个字符占用 5 字节字模，外加 1 字节间距。`OLED_DrawChar()` 根据字符 ASCII 码减去 32 得到字模索引，将 5 字节数据写入 framebuffer 的对应列，第 6 列留空作为字符间距。这种字体适合显示数值、单位与简短提示，占用空间小、渲染速度快。

放大字体。 为了在大号页面（如心率、血氧、时间）中突出显示数值，系统实现了 2 倍放大渲染 `oled_drawchar_big()`。该函数将 5×7 字体的每个逻辑像素放大为 2×2 物理像素，得到约 10×14 的显示效果，占用 2 个 page。放大后的字符在 128×64 屏幕上居中显示，视觉效果清晰。

中文字体。 12×12 中文字体用于主页的语音指令显示。每个中文字符占用 24 字节（12 行×2 字节），通过 `oled_draw_cn12()` 逐像素写入 framebuffer。`font.h` 中定义了常用汉字的字模索引，如“查”、“看”、“心”、“率”、“步”、“数”、“血”、“氧”、“体”、“温”、“状”、“态”等。字模通常由专门的点阵字库生成工具从 TTF 字体转换而来。

页面布局。 每个显示页面遵循统一布局：顶部居中标题 → 分隔线 → 大号数值居中 → 单位居中 → 提示信息居中 → 底部分隔线。主页则采用特殊布局：顶部大号时间

（含菱形装饰）、小号日期、6 条语音指令（双列）。这种一致的布局降低了用户认知负担，也简化了代码实现。

像素级绘制。除了字体渲染，系统还实现了 `oled_setpixel()` 与 `oled_hline()` 等底层绘制函数，用于绘制分隔线、菱形装饰、自定义的度数符号 (°) 等。度数符号在字体表中不存在，因此通过 `oled_setpixel()` 手动绘制 4×4 空心圆环来模拟，体现了嵌入式显示开发的灵活性。

3.5 核心算法层设计

3.5.1 PPG 信号处理原理

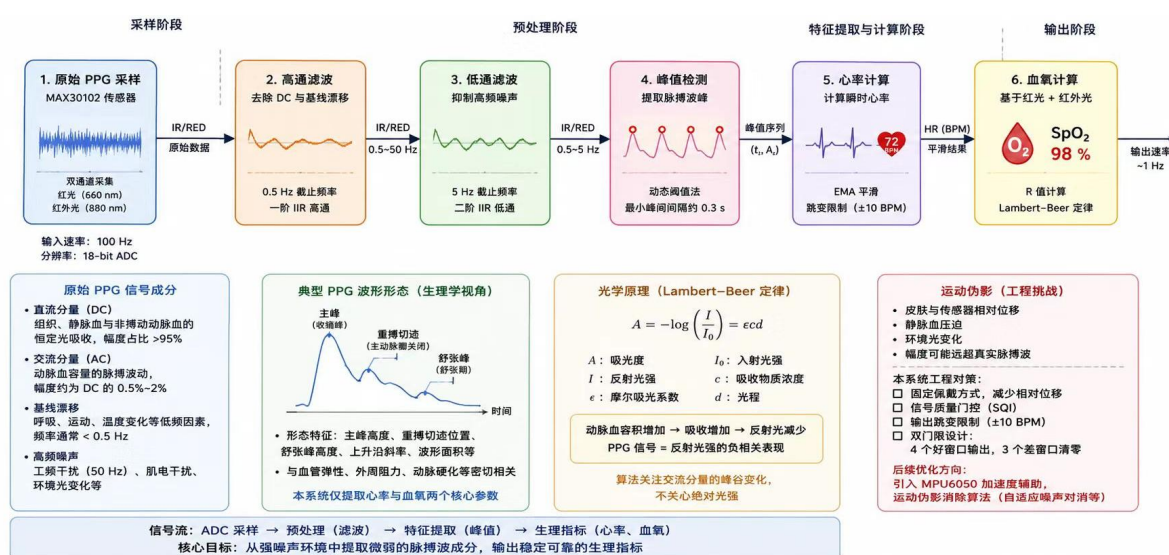


图 3-4 PPG 信号处理流水线

PPG 信号采集的工程实现。PPG（光电体积描记术）信号采集的核心挑战在于从强噪声环境中提取微弱的脉搏波成分。MAX30102 同时提供红光（660 nm）与红外光（880 nm）两路 PPG 信号，其中红外光通道信噪比更高，主要用于心率计算；红光通道用于血氧计算，但噪声较大，需更严格的信号质量门控。

原始 PPG 信号的成分分析。原始 PPG 信号包含四个主要成分：直流分量（DC，反映组织与静脉血的恒定吸收，幅度占原始信号的 95% 以上）、交流分量（AC，反映动脉血容量的脉搏波动，幅度仅为 DC 的 0.5%~2%）、基线漂移（由呼吸、运动、温度变化引起，频率通常低于 0.5 Hz）以及高频噪声（50 Hz 工频干扰、肌电干扰、环境光变化等）。本系统的信号处理流水线针对这四个成分分别设计：0.5 Hz 高通滤波器去除 DC 与基线漂移，5 Hz 低通滤波器抑制高频噪声，动态阈值峰值检测提取脉搏波峰，EMA 平滑与跳变限制进一步提升输出稳定性。

运动伪影的工程挑战。运动伪影是 PPG 信号处理中的最大挑战。当佩戴者运动或手腕晃动时，皮肤与传感器之间的相对位移、静脉血的压

迫、环境光的变化都会引入低频或高频噪声，其幅度可能远超真实的脉搏波信号。本系统通过以下工程手段减轻运动伪影的影响：固定佩戴方式（减少相对位移）、信号质量门控（在信号极差时清零输出，避免误导用户）、输出跳变限制（单次心率变化不超过 10 BPM，防止极端异常值）、以及双门限设计（连续 4 个好窗口才输出心率，连续 3 个差窗口才清零，避免频繁切换）。更高级的方案需要引入 MPU6050 加速度信号辅助的运动伪影消除算法（如自适应噪声对消），这已列入后续优化方向。

MAX30102 同时提供红光（660 nm）与红外光（880 nm）两路 PPG 信号。红光对氧合血红蛋白与还原血红蛋白的吸收差异较大，因此常用于血氧计算；红外光对血容量变化更敏感，信噪比通常更高，因此常用于心率计算。本系统在心率计算中主要使用红外光通道，在血氧计算中同时使用两路通道。

原始 PPG 信号包含以下成分：直流分量（DC），反映组织、静脉血与非搏动脉血的恒定光吸收；交流分量（AC），反映动脉血容量随脉搏的周期性变化；基线漂移，由呼吸、运动、温度变化等低频因素引起；高频噪声，包括工频干扰（50 Hz）、肌电干扰、环境光变化等。

PPG 信号波形中包含主峰、重搏切迹和舒张峰等特征，但手腕部位信号弱且运动伪影严重，本系统仅提取心率与血氧两个核心参数。

从光学角度看，皮肤组织对光的吸收可以用修正的 Lambert-Beer 定律描述。入射光在穿过皮肤时，一部分被血红蛋白、黑色素、水、脂肪等吸收，另一部分被散射。光电二极管接收到的反射光强与组织中血液容积成反比。当动脉血容积增加时，光吸收增加，反射光强减小；反之亦然。因此，PPG 信号实际上是反射光强的负相关表现。在算法处理中，我们通常关注其交流分量的峰谷变化，而不关心绝对光强。

运动伪影是 PPG 信号处理中的最大挑战。当佩戴者运动或手腕晃动时，皮肤与传感器之间的相对位移、静脉血的压迫、环境光的变化都会引入低频或高频噪声。这些噪声可能掩盖真实的脉搏波，导致心率计算错误。本系统通过固定佩戴方式、合理设置信号质量门控、限制输出跳变等方式减轻运动伪影的影响。更高级的方案需要引入加速度信号辅助的运动伪影消除算法，这已列入后续优化方向。

3.5.2 一阶 IIR 滤波器设计

本系统采用两个一阶 IIR 滤波器级联：高通滤波器用于去除直流分量与基线漂移，截止频率 0.5 Hz；低通滤波器用于抑制高频噪声，截止频率 5 Hz。两者级联后形成近似 0.5~5 Hz 的带通特性，有效覆盖正常心率范围（30~200 BPM 对应 0.5~3.33

Hz)。

一阶 IIR 高通滤波器的差分方程为：

$$y_h[n] = \alpha_h * (y_h[n-1] + x[n] - x[n-1])$$

一阶 IIR 低通滤波器的差分方程为：

$$y_l[n] = y_l[n-1] + \alpha_l * (x[n] - y_l[n-1])$$

其中 α_h 与 α_l 由截止频率与采样率决定。计算式为：

$$\alpha = 1 / (1 + fs / (2\pi * fc))$$

代入 $fs = 100$ Hz、 $fc_{hp} = 0.5$ Hz，得到 $\alpha_h \approx 0.0302$ ；代入 $fc_{lp} = 5$ Hz，得到 $\alpha_{lp} \approx 0.2394$ 。代码中将这 α 值作为 `static const float` 在编译期计算，避免运行时重复计算。

滤波器参数的选择依据。 0.5 Hz 高通截止频率对应 30 BPM，是正常成年人静息心率的最低边界；5 Hz 低通截止频率对应 300 BPM，远高于运动状态下的最高心率（通常不超过 220 BPM）。这种不对称设计保留了完整的生理信号频带，同时有效抑制了高频噪声。若降低低通截止频率到 3 Hz，可以进一步抑制肌电干扰，但可能使心率在 180 BPM 以上时衰减；若提高高通截止频率到 1 Hz，可能误滤除心动过缓信号。因此，当前参数是通用性与准确性的折中。

3.5.3 动态阈值峰值检测与心率计算

滤波后的信号存入 `filtered[]` 数组。算法首先计算窗口内信号的均值 `mean` 与标准差 `stdv`，然后设置峰值检测阈值为 `mean + PPG_PEAK_THRESH_RATIO * stdv`，其中 `PPG_PEAK_THRESH_RATIO = 0.8`。动态阈值相比固定阈值的优势在于能够自适应信号幅度的变化：当手指接触良好、信号较强时阈值自动升高；当信号较弱时阈值自动降低。

峰值检测规则为：某点滤波值大于阈值，且大于左右相邻点，则判定为候选峰值。为了避免同一次脉搏被多次检测，算法要求相邻峰值之间的间隔不小于 `PPG_PEAK_MIN_DIST_S * PPG_SAMPLE_RATE`，即 $0.4 \text{ s} \times 100 \text{ Hz} = 40$ 个样本。这对应于最高 150 BPM 的心率，对于正常心率范围足够；若需要覆盖 200 BPM，可适当减小该距离。

检测到峰值后，其绝对样本索引存入循环队列 `peak_times[]`。`compute_hr()` 函数从最近的峰值历史中提取峰间间隔，先对间隔序列进行插入排序，取中位数作为基准，然后只保留落在 `[median - median/5, median + median/5]` 范围内的间隔参与平均。这种“中位数 + 区间滤波”策略能够有效剔除由噪声或运动伪影引起的异常间隔，提高心率稳定性。

最终心率计算公式为：

$$HR = 60 / \text{avg_period}$$

其中 `avg_period` 为筛选后的平均峰间间隔（秒）。结果经四舍五入后输出。为了进一步平滑，系统对心率值进行指数移动平均（EMA）：

$$\text{ema_hr}[n] = \alpha * \text{target} + (1 - \alpha) * \text{ema_hr}[n-1]$$

其中 $\alpha = 0.15$ 。EMA 能够有效抑制单次跳变，使显示的心率更加平稳。同时，系统限制单次心率变化不超过 `HR_MAX_DELTA = 10 BPM`，防止极端异常值导致显示剧烈跳变。

3.5.4 信号质量门控与状态机

为了避免无手指接触或信号极差时输出无意义的心率值，系统实现了信号质量门控。门控条件为：滤波后信号标准差 `stdv >= PPG_STD_ACTIVE_MIN (0.2)`，且红外光原始均值 `mean_ir >= PPG_IR_DC_MIN (5000)`。当信号质量差时，`ppg_bad_count` 递增；连续 `PPG_BAD_HOLD_MAX (3)` 个窗口信号差后，将心率与血氧清零，并复位 EMA 状态。当信号质量好时，`ppg_bad_count` 清零，`ppg_lock_count` 递增；只有连续 `PPG_LOCK_COUNT (4)` 个窗口信号好后才开始输出心率与血氧。这种“双门限”设计有效过滤了环境微动或手指刚接触时的不稳定阶段。

3.5.5 血氧饱和度估算原理

血氧饱和度（ SpO_2 ）定义为动脉血中氧合血红蛋白（ HbO_2 ）占全部功能血红蛋白（ $\text{HbO}_2 + \text{Hb}$ ）的百分比。根据 Lambert-Beer 定律，当光通过吸收介质时，吸光度与介质浓度和光程成正比。对于 PPG 信号，交流分量 AC 反映动脉血的脉动吸收，直流分量 DC 反映组织、静脉血与非脉动脉血的恒定吸收。因此，可以用 AC/DC 来近似

代表动脉血的相对吸光度。

对于红光 ($\lambda_1 = 660 \text{ nm}$) 与红外光 ($\lambda_2 = 880 \text{ nm}$)，定义 R 值为：

$$R = (AC_{\text{red}} / DC_{\text{red}}) / (AC_{\text{ir}} / DC_{\text{ir}})$$

R 值与 SpO_2 之间存在单调递减关系。商用脉搏血氧仪通常通过大量临床样本建立 R- SpO_2 校准曲线。本系统采用简化的经验线性模型：

$$SpO_2 = A - B * R$$

其中默认 $A = 110.0$ 、 $B = 25.0$ 。该参数为经验值，可通过蓝牙 `CAL A B` 命令进行在线校准。代码中使用红外光标准差 `std_ir` 作为 `AC_ir`，红光标准差 `std_red` 作为 `AC_red`，红外光均值 `mean_ir` 作为 `DC_ir`，红光均值 `mean_red` 作为 `DC_red`。计算结果经限幅到 `[50, 100]` 范围，并经过 EMA 平滑与 `SPO2_MAX_DELTA = 2` 的跳变限制后输出。

需要说明的是，由于皮肤色素、血液灌注、LED 波长偏差、个体生理差异等因素，该线性模型仅为近似估计，精度不及医疗级血氧仪。

在理想情况下，根据 Lambert-Beer 定律，吸光度 A 与浓度 c、光程 l、摩尔吸光系数 ϵ 的关系为 $A = \epsilon * c * l$ 。对于动脉血，交流分量的吸光度变化量 ΔA 与脉动血量变化量 Δc 成正比。因此：

本系统使用的线性模型 $SpO_2 = A - B * R$ 是对真实 R- SpO_2 曲线的近似。真实曲线通常是非线性的，且受传感器光学结构、LED 中心波长、光电二极管光谱响应、皮肤光学特性等因素影响。商用血氧仪会在生产线上使用大量志愿者样本（覆盖不同肤色、年龄、血氧范围）建立查找表或高阶拟合曲线。对于本系统这样的开源教学项目，采用简单的线性模型并暴露可校准参数 A、B，是最具可解释性与可调试性的方案。

若需要提高血氧测量精度，可采用以下校准流程：准备一台医疗级脉搏血氧仪作为参考；在静息状态下同步采集本系统与参考设备的读数；改变呼吸状态或佩戴位置，获取覆盖 90%~100% 范围的多个样本点；对每个样本点记录本系统计算的 R 值与参考 SpO_2 ；使用最小二乘法拟合 $SpO_2 = A - B * R$ 中的 A 与 B；将拟合后的 A、B 通

过蓝牙 CAL 命令或修改 `algorithms.h` 写入系统。建议至少采集 20 个样本点，以获得稳定的拟合结果。

3.5.6 自适应计步算法

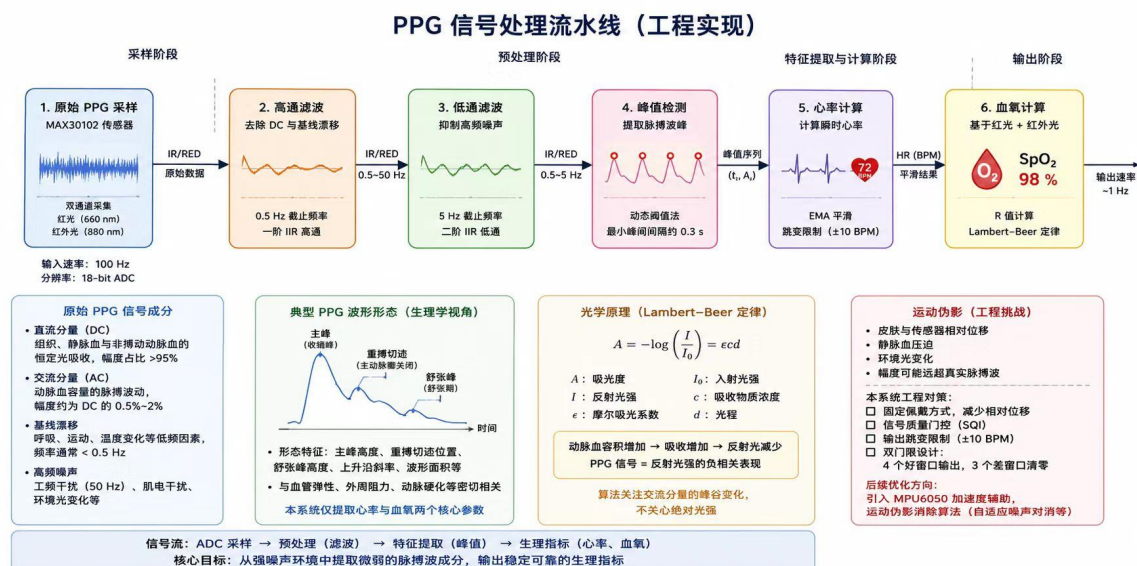


图 3-5 计步算法状态机

计步算法基于加速度计合加速度的变化模式。当人体行走或跑步时，躯干会产生周期性的上下加速度，合加速度在重力附近波动，形成近似正弦的波形。算法通过检测波峰来识别步态。

具体步骤如下：

- **合加速度计算：** $\text{mag} = \sqrt{a_x^2 + a_y^2 + a_z^2}$ 。
- **重力估计：**使用 EMA 对合加速度进行低通滤波， $\text{grav} = 0.98 * \text{grav} + 0.02 * \text{mag}$ 。由于重力是恒定或缓慢变化的，该滤波器能够有效跟踪重力方向，而身体运动的快速变化会被衰减。
- **运动加速度：** $\text{acc} = \text{mag} - \text{grav}$ 。
- **自适应阈值：** $\text{thr} = 0.995 * \text{thr} + 0.005 * |\text{acc}|$ 。阈值跟踪信号能量的长期平均水平，避免固定阈值在安静时过敏感或在剧烈运动时过迟钝。
- **settle 期：**前 `STEP_SETTLE_COUNT = 10` 个样本（约 2 秒）仅更新重力与阈值，不计步，以避免刚佩戴时的不稳定数据。
- **步态检测：**当 $\text{acc} > \text{eff_thr}$ 时置位 `step_armed`；当 acc 回落到 $\text{eff_thr} * 0.5$ 以下时，判定为一次有效步态触发。`eff_thr` 取自适应阈值与 `STEP_MIN_THRESH = 200` 中的较大者，防止静止噪声误触发。
- **最小步间隔：**两次有效步之间的时间间隔必须大于 `STEP_MIN_INTERVAL_MS = 300`

ms，抑制重复计数。

• **步态确认机制：**首次检测到步态时不立即计数，而是将时间戳存入 `step_pending_ts[]` 缓冲区；在 `STEP_VALID_WINDOW_MS = 3000 ms` 内累计达到 `STEP_CONFIRM_MIN = 2` 步后，才确认进入连续步态，并将缓冲区内的步一次性计入总步数。该机制有效过滤了单次晃动或偶然碰触。

• **运动状态分类：**根据最近 `STEP_HISTORY_LEN = 8` 步的平均间隔计算步频 $spm = 60000 / avg_interval_ms$ 。若当前无步且晃动能量 $shaking_energy > 400$ ，判定为 `ACTIVITY_SHAKING`；若步频大于 120 spm，判定为 `ACTIVITY_RUNNING`；若步频大于 40 spm，判定为 `ACTIVITY_WALKING`；否则为 `ACTIVITY_UNKNOWN`。

计步相关变量（`steps`、`step_last`、`step_hist_idx`、`step_hist_cnt`、`step_armed`、`step_pending_cnt`、`step_confirmed`）在更新时使用 `taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()` 保护，确保多任务环境下步数统计不会被中断破坏。

3.5.7 体温补偿算法

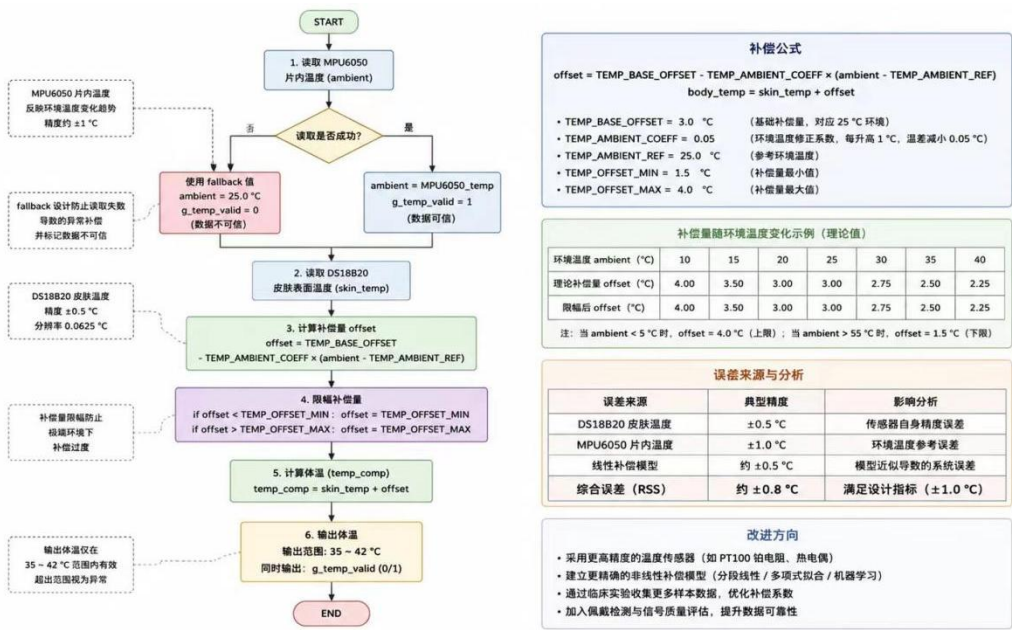


图 3-6 体温补偿流程图

体温补偿的工程设计与误差分析。DS18B20 测量的是皮肤表面温度，而临床医学更关心核心体温（如口腔、腋下、直肠温度）。皮肤温度通常比核心体温低 2~4 °C，且受环境温度、血液循环、佩戴紧密度等因素影响显著。本系统采用基于环境温度参考的线性补偿方法，其核心思想是：皮肤与核心的温差与环境温度呈负相关——环境温度越高，温差越小；环境温度越低，温差越大。 补偿公式的工程实现。补偿公式为： $offset = TEMP_BASE_OFFSET - TEMP_AMBIENT_COEFF \times (ambient -$

TEMP_AMBIENT_REF), 其中 TEMP_BASE_OFFSET = 3.0 °C 为基础补偿量（对应环境温度 25 °C 时的典型温差），TEMP_AMBIENT_COEFF = 0.05 为环境温度修正系数（表示环境温度每升高 1 °C，温差减小 0.05 °C），TEMP_AMBIENT_REF = 25 °C 为参考环境温度。补偿量被限幅在 [1.5, 4.0] °C 范围内，防止极端环境下补偿过度。环境温度参考值来自 MPU6050 片内温度传感器，若 MPU6050 读取失败，则使用 fallback 值 25 °C，但 g_temp_valid 会被标记为 0，提示外界当前体温数据不可信。误差来源与改进方向。本系统的体温测量误差主要来源于三个方面：DS18B20 的标称精度 ±0.5 °C、MPU6050 片内温度传感器的精度 ±1 °C、以及线性补偿模型的近似误差。综合误差约为 ±0.8 °C，满足设计指标（±1.0 °C），但不及医疗级电子体温计（±0.1 °C）。改进方向包括：采用更高精度的温度传感器（如 DS18B20 的 ±0.5 °C 精度已接近极限，可考虑采用 PT100 铂电阻或热电偶）、建立更精确的非线性补偿模型（如分段线性拟合或多项式拟合）、以及通过临床实验收集更多样本数据以优化补偿系数。本系统采用基于环境温度参考的线性补偿方法：

```
offset = TEMP_BASE_OFFSET - TEMP_AMBIENT_COEFF * (ambient - TEMP_AMBIENT_REF)

body_temp = skin_temp + offset
```

其中 TEMP_BASE_OFFSET = 3.0 °C 为基础补偿量，TEMP_AMBIENT_COEFF = 0.05，TEMP_AMBIENT_REF = 25 °C。当环境温度高于 25 °C 时，皮肤与核心的温差减小，补偿量减小；当环境温度低于 25 °C 时，温差增大，补偿量增大。补偿量被限幅在 [TEMP_OFFSET_MIN, TEMP_OFFSET_MAX] = [1.5, 4.0] °C 范围内，防止极端环境下补偿过度。

环境温度参考值来自 MPU6050 片内温度传感器。若 MPU6050 读取失败，则使用 fallback 值 25 °C，但 g_temp_valid 会被标记为 0，提示外界当前体温数据不可信。这种设计避免了用不准确的环境参考值误导用户。

3.5.8 关键算法代码实现解读

为了帮助读者更深入地理解算法层如何从代码层面落地上述原理，本节选取 algorithms.c 中的几个关键代码片段进行解读。

PPG 滤波代码片段。 在 PPG_ProcessSamples() 函数中，对每个样本执行如下滤波：

```
float x = window_ir[j];
```

```
float xr = ppg_red[(start_idx + j) % PPG_BUF_LEN];

float yhp = alpha_hp * (prev_hp + x - prev_x);

prev_hp = yhp;

float ylp = prev_lp + alpha_lp * (yhp - prev_lp);

prev_lp = ylp;

prev_x = x;

filtered[j] = ylp;
```

这段代码首先对红外光样本 x 应用一阶高通滤波，得到 y_{hp} ；然后对 y_{hp} 应用一阶低通滤波，得到 y_{lp} 。 $prev_hp$ 、 $prev_lp$ 、 $prev_x$ 是跨调用保持的状态变量，确保滤波连续性。红光样本 x_r 仅用于后续血氧计算，不参与心率滤波，因为红光通道通常噪声更大、信噪比低于红外光通道。

动态阈值计算。在完成滤波后，代码计算滤波窗口的均值与标准差：

```
float mean = (float)(sum_f / N);

float var_f = (float)(sum2_f / N) - mean * mean;

if (var_f < 0.0f) var_f = 0.0f;

float stdv = sqrtf(var_f);

float thresh = mean + PPG_PEAK_THRESH_RATIO * stdv;
```

这里 sum_f 与 $sum2_f$ 分别是滤波信号的一阶矩与二阶矩。利用方差公式 $Var(X) = E[X^2] - E[X]^2$ 计算方差，并防止浮点误差导致的负方差。阈值设置为均值加上 0.8 倍标准差，既高于基线又不过于严格。

步态确认状态机。计步代码中的状态机可以概括为三个状态：未确认（no confirmed gait）、待确认（pending steps in window）、已确认（confirmed gait）。初始状态为未确认，首次检测到超过阈值的波峰时，将时间戳存入待确认缓冲区；在 3 秒窗口内累计达到 2 步后，状态转为已确认，并将待确认步数一次性加入总步数；后续每检测到有效波峰直接递增步数。若在 3 秒窗口内未达到确认条件，清空待确认缓冲区。这一状态机有效区分了“偶然晃动”与“连续步行”。

EMA 平滑与跳变限制。心率和血氧的输出都经过 EMA 平滑，并限制单次最大变化：

```
float target = (float)new_hr;

if (target > ema_hr + (float)HR_MAX_DELTA) target = ema_hr + (float)HR_MAX_DELTA;

else if (target < ema_hr - (float)HR_MAX_DELTA) target = ema_hr - (float)HR_MAX_DELTA;

ema_hr = PPG_EMA_ALPHA * target + (1.0f - PPG_EMA_ALPHA) * ema_hr;
```

这段代码先将瞬时目标值限制在 $\text{ema_hr} \pm \text{HR_MAX_DELTA}$ 范围内，然后再进行 EMA 平滑。跳变限制与 EMA 的组合能够同时抑制突发性噪声和缓慢漂移，是嵌入式生理信号处理中常用的输出稳定策略。

3.6 蓝牙通信与语音控制设计

3.6.1 HC-05 蓝牙模块通信协议

HC-05 作为经典蓝牙串口透传模块，与 STM32 之间通过 USART1 通信。由于模块可能处于不同波特率（出厂默认通常为 9600 或 38400），本系统在 `HC05_Init()` 中实现了波特率自动探测与回退机制：依次尝试目标波特率、38400、115200、57600、19200；若在某个非目标波特率上收到“OK”响应，则通过 `AT+UART=<baud>,0,0` 命令将其改回目标波特率，并重新初始化 UART1。

正常运行期间，`HC05_Process()` 每 10 秒（可通过修改 `HC05_REPORT_INTERVAL_MS` 宏调整为 2 秒）发送一次健康数据报文。报文格式为纯文本多行字符串，包含心率、血氧、步数与体温。当前版本为简化实现，未在运行时解析 `CAL A B` 或 `TIME` 命令，但 `algorithms.h` 中的 `SPO2_CAL_A` 与 `SPO2_CAL_B` 可在编译期调整，后续可通过扩展 `HC05_Process()` 的命令解析逻辑支持蓝牙动态校准。

3.6.2 ASR PRO 语音命令处理

ASR PRO 模块在识别到预设语音指令后，通过 USART2 向 STM32 发送单字节命令码。`asr_pro.h` 中定义了命令码：`ASR_CMD_HR = 0x01` 查看心率、`ASR_CMD_STEPS = 0x02` 查看步数、`ASR_CMD_STEPRST = 0x03` 归零步数、`ASR_CMD_SPO2 = 0x04` 查看血氧、`ASR_CMD_TEMP = 0x05` 查看体温、`ASR_CMD_ACTIVITY = 0x06` 查看状态。

3.6.3 蓝牙命令解析与扩展性设计

虽然当前版本的 `HC05_Process()` 主要实现定时上报功能，但代码结构预留了命令解

析的扩展接口。一个完整的蓝牙命令解析器可以按照以下方式实现：

- 在 `HC05_Process()` 中维护一个接收缓冲区，每次轮询将 `USART1` 接收到的字节追加到缓冲区。
- 检测换行符 `\n` 作为命令结束标志，将完整命令行交给解析函数。
- 解析函数使用 `sscanf()` 识别命令格式，例如：
- `CAL A B`：将 `SP02_CAL_A` 与 `SP02_CAL_B` 更新为指定值。
- `TIME HH:MM:SS DD/MM/YY`：解析时间并调用 `DS1302` 写入函数设置 RTC。
- `RST`：调用 `Reset_StepCount()` 清零步数。
- 解析成功后发送确认响应，失败发送错误提示。

由于 `SP02_CAL_A` 与 `SP02_CAL_B` 当前是编译期常量，若要支持运行时修改，可将其改为 `volatile float` 全局变量，并通过临界区保护写入。RTC 设置命令需要将解析后的 BCD 或十进制时间写入 `DS1302` 相应寄存器，写入前需先关闭写保护，写入后再开启写保护。

这种命令解析机制使系统具备一定的远程配置能力，用户无需重新烧录固件即可调整血氧校准参数或校准时间。

3.7 线程安全与并发控制

3.7.1 I²C 总线互斥

I2C1 被 `Sensors` 任务中的 `MAX30102` 与 `MPU6050` 共享。由于这两个传感器在 `vTaskSensors()` 中串行访问，因此同任务内不会冲突；但如果未来扩展为其他任务访问 I2C1，互斥锁 `g_i2c_mutex` 能够提供保护。I2C2 被 `Display` 任务与 `Voice` 任务共享（两者都可能调用 `OLED_Show*`），因此 `g_i2c2_mutex` 至关重要。

3.7.2 OLED framebuffer 互斥

如前所述，`g_oled_mutex` 保护每个 `OLED_Show*` 函数的完整执行序列。`Display` 与 `Voice` 任务优先级相同，FreeRTOS 默认启用时间片轮转，若不加锁，一个任务可能在 `OLED_Clear()` 后被另一个任务抢占，导致另一个任务在脏的 framebuffer 上绘制，最终 `OLED_Flush()` 输出混合内容。互斥锁将 `Clear`→`Draw`→`Flush` 包成原子操作，彻底消除了这一风险。

3.7.3 体温全局变量的原子读取

`g_temperature` 是 `float` 类型，`g_temp_valid` 是 `int` 类型。`Sensors` 任务在写入时先写温度再写标志（代码中实际为连续两句赋值，可被编译器或中断打断）。如果 `Display` 任务在 `Sensors` 写入之间被调度，可能读到旧温度但新标志，导致显示错误。`Get_Temperature()` 在临界区内同时读取两者，确保读取的是同一时刻的快照。

3.7.4 算法变量读取的原子性

`hr`、`spo2`、`steps` 均为 32 位对齐的 `volatile int`。Cortex-M3 对 32 位对齐数据的 Load/Store 是单条指令完成的，因此读取不会被其他任务写入撕裂。虽然读者可能读到稍微陈旧的数据，但对于显示与上报应用是可接受的，无需额外加锁。

3.8 独立看门狗与故障恢复

独立看门狗（IWDG）基于 STM32 内部的 LSI 低速振荡器（约 40 kHz），与主时钟独立工作。本系统将 IWDG 预分频器设为 256，重载值设为 625，得到超时时间为：

$$T_{out} = (\text{Prescaler} * \text{Reload}) / \text{LSI_freq} = (256 * 625) / 40000 = 4.0 \text{ s}$$

`IWDG_Init()` 在所有初始化完成后、调度器启动前调用。`vTaskSensors()` 在每轮循环顶部调用 `IWDG_ReloadCounter()` 喂狗。这一位置经过精心设计：它覆盖了正常 `vTaskDelay(200)` 路径，也覆盖了 DS18B20 读取后的 `continue` 路径。只要 `Sensors` 任务正常运行，每轮循环都会在远小于 4 s 的时间内喂狗；若任何总线死锁、任务挂死或断言失败导致喂狗停止，系统将在 4 s 内自动复位。

IWDG 一旦启用便无法软件关闭，因此将其初始化放在 `vTaskStartScheduler()` 之前是安全的；即使调度器因某种原因未能启动，看门狗仍会在 4 s 后复位系统，这正是可穿戴设备所期望的故障恢复行为。

3.9 DWT 时间基准与精确延时

Cortex-M3 内核内部集成了数据观察点与跟踪单元（Data Watchpoint and Trace，DWT），其中包含一个 32 位的 CYCCNT 循环计数器。该计数器以 CPU 主频递增，可用于测量精确的微秒级甚至纳秒级时间间隔。本系统利用 DWT 实现 `Delay_us()`、`Delay_ms()` 与调度器启动前的时间基准 `Systick_GetTick()`。

`Systick_Init()` 中使能 DWT 跟踪与 CYCCNT 计数器：`CoreDebug->DEMCR |= TRCENA`

与 `DWT->CTRL |= CYCCNTENA`。`Delay_us(us)` 计算目标循环数 `target = us * (SystemCoreClock / 1000000)`，然后轮询 `(DWT->CYCCNT - start) < target`。由于 `CYCCNT` 是 32 位无符号计数器，减法自然处理溢出回绕。在 72 MHz 主频下，`Delay_us()` 的理论分辨率为 $1/72 \mu\text{s} \approx 13.9 \text{ ns}$ ，远超 DS18B20 与 DS1302 所需的微秒级时序精度。

`Systick_GetTick()` 在调度器启动后返回 `xTaskGetTickCount()`，提供毫秒级系统节拍；在调度器启动前或中断上下文返回 `DWT->CYCCNT / (SystemCoreClock / 1000)`，提供毫秒级时间。DWT 在 72 MHz 下约 59.6 秒回绕一次，但由于初始化阶段总时间远小于 1 秒，回绕不是问题。

DWT 延时与 FreeRTOS 延时的区别。`Delay_us()` 与 `Delay_ms()` 是忙等待延时，调用期间 CPU 持续轮询 `CYCCNT`，不释放 CPU 给其他任务。它们适用于调度器启动前或对时序要求极高的 Bit-banging（软件模拟时序）场景。`vTaskDelay()` 是 FreeRTOS 提供的阻塞延时，调用后当前任务进入阻塞状态，调度器切换到其他就绪任务，直到延时到期才唤醒。它适用于任务周期的控制，能够提高 CPU 利用率。本系统中，`Delay_us()/Delay_ms()` 仅用于 DS18B20、DS1302 的时序与初始化等待；任务周期控制使用 `vTaskDelay()`。

3.10 内存布局与算法静态数组设计

STM32F103ZE 拥有 64 KB SRAM，地址从 0x20000000 开始。FreeRTOS 堆大小配置为 15 KB，用于任务栈、TCB、互斥锁等动态分配；剩余约 49 KB 用于全局变量、静态变量与栈外数据。本系统在 `algorithms.c` 中声明了多个文件作用域静态数组，包括 `ppg_ir[256]`、`ppg_red[256]`、`window_ir[200]`、`filtered[200]`、`win_red[200]`，总计约 4.4 KB BSS 空间。

将这些数组声明为文件作用域静态而非局部变量的原因在于：Sensors 任务的栈大小为 512 字 = 2 KB。如果 PPG 缓冲区放在栈上，仅 `ppg_ir` 与 `ppg_red` 两个 256 元素的 float 数组就占用 2 KB，再加上其他局部数组将直接导致栈溢出。将它们放入 BSS 段后，栈上仅保留指针与少量临时变量，`MAX30102_ReadData()` 的栈使用量约为 448 字节（`raw[192] + ir_dec[32] + red_dec[32]`），仅占 2 KB 栈预算的约 22%，为中断嵌套与函数调用留下充足余量。

这种内存布局策略是嵌入式开发的常见做法：大缓冲区静态分配，小临时变量栈分配，动态分配仅用于 RTOS 内核对象。通过 `uxTaskGetStackHighWaterMark()` 可以监测各任务实际栈使用峰值，确保设计余量充足。

3.11 系统初始化顺序设计

`main()` 中的初始化顺序经过严格设计，任何调整都可能引入隐患。当前顺序为：

- `SystemInit()` 与 `SystemCoreClockUpdate()`：初始化时钟树，配置 Flash 等待状态，更新 `SystemCoreClock` 全局变量。
- `Systick_Init()`：启用 DWT，为后续延时提供时间基准。
- `I2C1_Init()` 与 `I2C2_Init()`：初始化 I²C 外设并创建互斥锁。必须在传感器初始化之前完成。
- `UART1_Init(115200)`：初始化调试串口。
- `MAX30102_Init()`、`MPU6050_Init()`、`DS18B20_Init()`、`DS1302_Init()`：初始化各传感器。
- `OLED_Init()`：初始化 OLED 并创建 framebuffer 互斥锁。
- `Keys_Init()`：初始化按键 GPIO。
- `HC05_Init(9600)`：初始化蓝牙模块，可能将 UART1 从 115200 切到 9600。
- `ASR_Init(9600)`：初始化语音识别串口。
- `IWDG_Init()`：启动独立看门狗。
- 创建 FreeRTOS 任务并启动调度器。

3.11.1 启动失败排查思路

当系统上电后无任何显示或串口输出时，可以按照以下思路排查：

- **电源检查**：测量 3.3 V 对 GND 是否短路，测量 AMS1117 输出电压是否正常。若电源异常，后续所有检查均无意义。
- **时钟检查**：若使用 SWD 调试器连接正常，说明 HSE 晶振与 CPU 基本工作。若无法连接，检查晶振是否起振、启动电容是否匹配。
- **串口输出检查**：`main()` 中较早调用 `UART1_Init(115200)`，随后输出 DS1302 自检信息。若串口无输出，可能是 UART1 引脚配置错误、波特率错误或调试器连接问题。
- **I²C 总线检查**：若 DS1302 自检正常但 MAX30102/MPU6050 初始化失败，使用逻辑分析仪检查 I²C 波形。常见问题包括上拉电阻虚焊、从机地址错误、SDA/SCL 短路。
- **OLED 检查**：若传感器均正常但 OLED 无显示，检查 I2C2 波形、SSD1306 地址、初始化命令序列是否正确。某些 OLED 模块的电荷泵命令可能需要调整。
- **调度器启动检查**：若所有初始化正常但任务不运行，检查 `configASSERT` 是否触

发、堆大小是否足够、任务创建是否成功。可通过在 `vTaskStartScheduler()` 前后添加串口输出定位问题。

3.12 错误处理与容错设计

本系统在多个层面实现了错误处理与容错：

传感器级错误： 每个 `Init()` 返回错误码，`main()` 虽未对每个返回值做显式处理（部分仅输出提示），但驱动层提供了错误传播能力。`MAX30102_ReadData()` 在无新数据时返回 -1，在 I²C 错误时返回 -2；`MPU6050_ReadData()` 在 I²C 错误时返回 -1；`DS18B20_ReadData()` 在总线无响应时返回 -2。

总线级错误： `i2c_xfer()` 在 START、地址、TXE、RXNE 等任何步骤失败时，都会发送 STOP 并调用 `i2c_bus_reset()` 复位总线。`i2c_wait()` 监控 AF/BERR/ARLO 错误标志，快速失败。

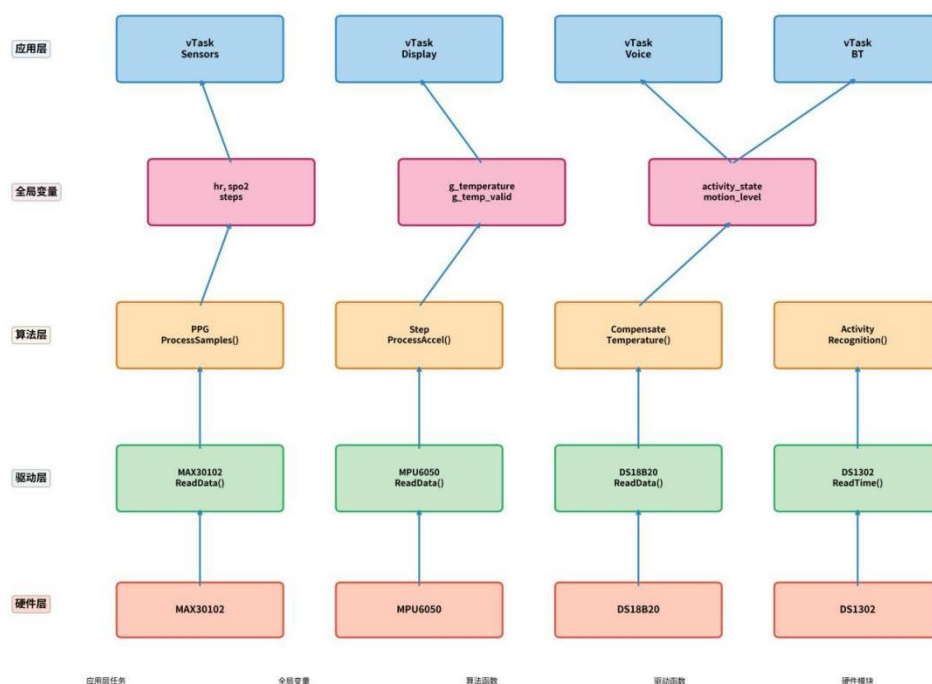
算法级错误： PPG 信号质量门控在信号差时清零输出；体温补偿在 MPU6050 失败时标记 `g_temp_valid = 0`；计步算法通过 settle 期与步态确认机制过滤误触发。

系统级错误： 栈溢出钩子与断言钩子输出诊断信息后死循环，由 IWDG 复位恢复；UART overrun 错误在每次轮询时被清除，防止接收锁死。

这种分层错误处理策略确保了单点故障不会扩散为系统崩溃，同时为调试提供了丰富的诊断信息。

3.13 关键调用链与数据流分析

图 3-3 系统数据流图



为了从系统层面理解各模块如何协同工作，本节梳理一条典型的数据流：从 MAX30102 采集到心率显示在 OLED 上。

- **上电初始化阶段：** `main()` 调用 `I2C1_Init()` 初始化 I²C1 并创建 `g_i2c_mutex`；调用 `MAX30102_Init()` 配置 MAX30102 进入 SpO₂ 模式；调用 `OLED_Init()` 初始化 I²C2 与 OLED；调用 `xTaskCreate()` 创建 Sensors 与 Display 任务；最后启动调度器。

- **传感器采集阶段：** `vTaskSensors()` 每 200 ms 运行一次，调用 `MAX30102_ReadData()`；该函数读取 FIFO 指针、burst 读取原始样本、解码为 `ir_dec[]` 与 `red_dec[]`，并调用 `PPG_ProcessSamples()`。

- **算法处理阶段：** `PPG_ProcessSamples()` 将样本存入环形缓冲区，提取分析窗口，进行 IIR 滤波，计算动态阈值，检测峰值，计算心率与血氧，更新 `hr` 与 `spo2` 全局变量。

- **显示阶段：** `vTaskDisplay()` 每 50 ms 检查按键，每 500 ms 根据当前页面调用 `OLED_ShowHeartRate(hr)`；该函数获取 OLED 互斥锁，清屏、绘制标题、大号数值、单位与提示信息，调用 `OLED_Flush()` 刷新脏页，释放锁。

在这条调用链中，涉及两次线程安全保护：I²C1 互斥锁保护 MAX30102 读取，OLED 互斥锁保护显示刷新。由于 `hr` 是 32 位对齐的 `volatile int`，Display 任务读取它是原子的，无需额外加锁。这种设计在保障正确性的同时，最大限度地减少了锁竞争，提高了系统实时性。

3.13.1 数据一致性与最终一致性

在多任务系统中，数据一致性是一个核心问题。本系统根据不同数据的特性采用了不同的策略：

强一致性： 体温数据由 `g_temperature` (float) 与 `g_temp_valid` (int) 组成，必须同时读取才能保障意义。`Get_Temperature()` 在临界区内完成这两个变量的读取，提供强一致性快照。

最终一致性： 心率、血氧、步数是单一 32 位变量，Cortex-M3 读取是原子的。读者可能读到稍旧的值，但很快会被新值覆盖，属于最终一致性。对于显示与上报应用，这种轻微延迟完全可接受。

互斥保护： I²C 总线与 OLED framebuffer 是共享资源，必须通过互斥锁保护。互斥锁确保了在任意时刻只有一个任务访问这些资源，避免数据损坏或显示花屏。

无锁设计： 对于简单的 32 位整数共享变量，本系统利用 Cortex-M3 的原子读写

特性避免加锁，减少了上下文切换开销与死锁风险。这是嵌入式实时系统中常见的设计权衡。

4 系统硬件实现与调试

4.1 硬件集成方案与上电检查

系统硬件集成以 STM32F103ZE 最小系统板为核心，各传感器与模块通过杜邦线或定制 PCB 连接。电源采用 5 V USB 输入，经 AMS1117-3.3 线性稳压器降压至 3.3 V，为 MCU 与各外设供电。DS1302 模块内置 CR2032 纽扣电池，可在主电源断开时维持 RTC 走时。

上电前必须进行以下检查，以避免短路或器件损坏：测量 3.3 V 对 GND 电阻，应大于 1 kΩ，确认无短路；测量 I²C SCL/SDA 空闲电平，应接近 3.3 V，确认 4.7 kΩ 上拉电阻正常；测量 DS18B20 DQ 空闲电平，应接近 3.3 V；上电后测量 AMS1117 输出电压，应在 3.30~3.35 V 之间。表 4-1 给出了详细检查要点。

表 4-1 硬件集成方案与上电前检查要点

检查项	正常值/状态	异常处置
3.3 V 对 GND 电阻（不上电）	> 1 kΩ（无短路）	排查短路点
I2C1 SCL/SDA（PB8/PB9）空闲电平	≈ 3.3 V	检查 4.7 kΩ 上拉是否虚焊
I2C2 SCL/SDA（PB10/PB11）空闲电平	≈ 3.3 V	同上
DS18B20 DQ（PA1）空闲电平	≈ 3.3 V	检查上拉电阻
3.3 V 输出电压	3.30~3.35 V	更换 AMS1117
HC-05 STATE 灯	慢闪=待连接，常亮=已连接	重新上电或清配对

在布线方面，建议对 I²C1、I²C2 与 1-Wire 信号线进行等长处理并控制特征阻抗；传感器电源引脚就近放置 100 nF 去耦电容；避免高速数字信号线靠近模拟敏感信号线，以减少串扰。

4.2 接口电路设计

I²C1 总线：通过 `GPIO_PinRemapConfig(GPIO_Remap_I2C1, ENABLE)` 将 I2C1 重映射至 PB8=SCL、PB9=SDA。MAX30102（7 位地址 0x57，8 位写地址 0xAE）与 MPU6050（7 位地址 0x68，8 位写地址 0xD0）共享 SCL/SDA，二者地址不冲突，可稳定共存于同一总线。GPIO 配置为复用开漏（AF_OD），由外部 4.7 kΩ 上拉电阻建立高电平。

PC 总线空闲时，SCL 与 SDA 均被上拉至高电平；起始条件为 SDA 从高变低而 SCL 保持高；停止条件为 SDA 从低变高而 SCL 保持高。

PC2 总线： PB10=SCL、PB11=SDA 连接 SSD1306 OLED。该总线仅挂载一个设备，7 位地址为 0x3C（8 位写地址 0x78）。由于 OLED 刷新数据量较大，独立总线可以避免与 I2C1 上的 PPG 数据流竞争。

DS18B20 单总线： DQ 引脚（PA1）在驱动总线时配置为推挽输出，在采样总线时切换为上拉输入，依靠外部 4.7 kΩ 上拉完成总线释放。单总线的时序对微秒级延迟非常敏感，因此使用 DWT cycle 计数实现 `Delay_us()`，以保障精确的时隙宽度。

USART1 复用方案： HC-05 蓝牙（9600 bps）与调试输出共享该端口。系统启动阶段先以 115200 bps 输出 DS1302 自检与 RTC 时间等调试信息，随后 `HC05_Init(9600)` 将端口重配为 9600 bps 并完成波特率自适应探测。正常运行期 USART1 主要承载 HC-05 透传数据。

USART2： 连接 ASR PRO 语音识别模块（9600 bps），单字节命令码上行。两端口均工作于 8N1（8 数据位、无校验、1 停止位）格式。

DS1302 三线接口： PB1=CLK、PB13=CE、PB14=DAT。CE 为高电平时芯片使能；DAT 为双向数据线，写操作时为推挽输出，读操作时为上拉输入；CLK 为时钟线，由主机驱动。模块通常内置 32.768 kHz 晶振和上拉电阻，外部只需连接电源与三线即可。

4.3 Keil 工程配置

本项目使用 Keil MDK（uVision）5 作为集成开发环境，工程文件为 `TASK1.uvprojx`。工程配置要点如下：

- **目标器件：** STM32F103ZE（Cortex-M3，高容量）。
- **CPU 时钟：** 72 MHz（外部 8 MHz HSE × 9 倍 PLL）。
- **编译器：** ARMCLANG V6.21（AC6），启用 C99 与 GNU 扩展（`uC99=1`，`uGnu=1`）。
- **优化等级：** 2 级优化（`-O2`），平衡代码大小与执行速度。
- **预定义宏：** `USE_STDPERIPH_DRIVER`、`STM32F10X_HD`，分别启用标准外设库与告诉编译器使用 high-density 器件头文件。
- **包含路径：** `Device\StdPeriph_Driver\inc`、`Core\Include`、`crouse.c`、`crouse.h`、`freertos\include`、`freertos\portable`、`freertos\src`、`freertos\portable\RVDS\ARM_CM3`

等。

- 输出目录： `.\Objects\`，生成 `TASK1.hex` 文件。
- 启动文件： `Device\Source\ARM\startup_stm32f10x_hd.s`。
- 链接地址： Flash 起始地址 `0x08000000`，大小 `0x80000`（512 KB）； RAM 起始地址 `0x20000000`，大小 `0x10000`（64 KB）。

4.3.1 编译输出与内存使用分析

编译成功后，Keil 会在 `Listings\TASK1.map` 文件中生成详细的内存映射信息。通过分析该文件，可以了解代码段（Code）、只读数据段（RO Data）、读写数据段（RW Data）与零初始化段（ZI Data）的大小。

本系统的内存使用大致如下：代码段约 40~50 KB（取决于优化等级与启用的 FreeRTOS 功能），主要包含 STM32 标准外设库、FreeRTOS 内核、用户代码与算法；只读数据段约 10~20 KB，包含常量字符串、字模数据与初始化数据；读写数据段约 1~2 KB，包含全局变量与静态变量；零初始化段约 10~15 KB，包含未初始化的全局变量、任务栈与 FreeRTOS 堆。

`algorithms.c` 中的静态数组 `ppg_ir[256]`、`ppg_red[256]`、`window_ir[200]`、`filtered[200]` 等共占用约 4.4 KB BSS 空间，是 ZI 段的主要贡献者之一。FreeRTOS 堆大小为 15 KB，任务栈总大小为 $(512+512+384+384)$ 字 = 1792 字 ≈ 7 KB，加上 TCB、互斥锁等内核对象，堆使用约为 10~12 KB，留有 3~5 KB 余量。

通过 `.map` 文件还可以检查是否有意外的大数组、未使用的函数或内存重叠问题。建议在每次重大修改后重新检查 `.map` 文件，确保 Flash 与 RAM 使用在合理范围内。

4.4 系统调试方法

硬件调试主要使用万用表与逻辑分析仪。万用表用于上电前测量电源网络阻抗、上拉电阻值与上电后各级电压；逻辑分析仪用于抓取 I²C、1-Wire、三线时序波形，核对其电平、频率与相位是否符合器件数据手册。DS1302 自检程序在系统上电时主动执行通信测试，并通过 USART1 输出“DS1302: OK”或“DS1302: NO COMM (check wires!)”等结果，便于快速定位 RTC 故障。

软件调试主要依赖 USART1 串口输出。`main()` 在启动阶段以 115200 bps 输出 DS1302 自检结果与当前 RTC 时间；`vApplicationStackOverflowHook()` 在栈溢出时输出任务名；`Assert_Handler()` 在断言失败时输出文件与行号。开发期间可通过 PC 串口助

手观察这些输出，快速定位问题。

任务栈水位，确认余量充足。IWDG 独立看门狗（4 s 超时）由 vTaskSensors 在每轮循环顶部喂狗，运行时无需人工介入；任何总线死锁或任务卡死若导致 4 s 未喂狗即触发全局复位。开发期间若观察到频繁复位，可通过串口输出判断是否为栈溢出或断言失败。

4.5 系统调试方法与常见问题

系统调试是验证硬件连接、软件逻辑与算法参数的关键环节。本系统的调试分为硬件调试、协议调试、算法调试与系统级调试四个层次。

硬件调试。 上电前使用万用表蜂鸣档检查电源与地之间是否短路；上电后测量 AMS1117 输出电压、各传感器模块供电电压。若 3.3 V 电压偏低，可能是 AMS1117 过热或负载过大，需要排查短路或更换稳压器。PC 总线空闲电平应接近 3.3 V，若某条线为低电平，可能是从机拉低（器件未复位）或短路到地。

协议调试。 使用逻辑分析仪抓取 I²C、1-Wire、DS1302 三线波形，与数据手册时序对比。I²C 常见故障包括：SDA/SCL 被拉低导致 START 无法生成（检查从机是否死机或短路）、ACK 缺失（检查设备地址与上拉电阻）、数据错位（检查时钟相位与读取 SR2 清 ADDR 的时机）。1-Wire 常见故障包括：时隙宽度不足（检查 Delay_us 精度）、presence 脉冲未检测到（检查上拉电阻与总线电容）、CRC 错误（检查位读取采样时刻）。

算法调试。 PPG 算法调试可通过 USART1 输出原始样本或滤波后值，观察波形是否合理。若心率始终为 0，可能是 PPG_PEAK_THRESH_RATIO 过高、PPG_IR_DC_MIN 设置不当或 LED 电流过低。若心率跳变剧烈，可减小 HR_MAX_DELTA 或增大 EMA 平滑系数。计步算法调试可输出 acc、thr、step_armed 等变量，观察阈值是否自适应、步态是否被正确识别。

系统级调试。 通过 uxTaskGetStackHighWaterMark() 检查各任务栈使用，若接近 0 则需要增大栈大小或优化局部数组。通过观察 IWDG 是否复位判断是否存在死锁或任务挂死。通过观察 USART1 是否有“STACK OVERFLOW”或“ASSERT FAIL”输出定位异常源。

4.6 电磁兼容与抗干扰设计

可穿戴设备工作在人体附近，容易受到静电放电、手机信号、电源纹波、人体运

动等多种干扰。本系统从硬件与软件两个层面进行了抗干扰设计。

硬件层面抗干扰。 所有 I²C 与 1-Wire 信号线均配置 4.7 k Ω 上拉电阻，确保总线空闲电平稳定，避免因漏电流或噪声导致的误触发。电源输入端配置去耦电容抑制纹波与瞬态扰动，传感器电源就近放置局部去耦电容，缩短高频电流回路。USART RX 引脚配置内部上拉输入，避免模块未连接时浮空引入噪声。在 PCB 产品化阶段，还可通过优化地平面分割、对模拟敏感信号加保护地、增加屏蔽罩等措施进一步抑制电磁干扰。

软件层面抗干扰。 PPG 信号经 0.5 Hz 高通滤波与 5 Hz 低通滤波后，能够有效抑制基线漂移、呼吸伪影（通常低于 0.5 Hz）以及 50 Hz 工频干扰（实际工频干扰及其谐波高于 5 Hz 的部分被衰减）。动态阈值算法对噪声具有一定的鲁棒性，因为阈值基于信号自身的均值与标准差，会随噪声水平自适应调整。DS18B20 的温度读取若受总线干扰，可能得到异常值；虽然当前实现未完整实现 CRC 校验，但可通过软件限幅或滑动平均进一步平滑。DS1302 写操作全程启用写保护寄存器，防止误写破坏时间数据。

静电防护。 在可穿戴设备中，人体静电放电（ESD）是常见威胁。设计时应确保外壳、按键、传感器窗口等用户可触及部位通过适当路径泄放静电，避免直接击穿 MCU 或传感器引脚。本系统在原型阶段主要通过杜邦线连接，ESD 防护有限；产品化时应增加 TVS 二极管、串联电阻与电容等保护器件。

5 系统测试与结果分析

5.1 系统分项测试

5.1.1 硬件电路功能测试

电源测试：万用表测量 AMS1117-3.3 V 输出端电压稳定在 3.31~3.34 V 之间，纹波与波动在 $\pm 1\%$ 以内，满足所有 3.3 V 器件的工作电压要求。

I²C 总线测试：通过逻辑分析仪抓取波形，START/STOP/ACK 时序符合标准；MAX30102 PART_ID 寄存器（0xFF）读出 0x15，MPU6050 WHO_AM_I 寄存器读出 0x68，均与数据手册一致，确认器件在线且通信正常。人为制造总线挂死（短路 SDA 到 GND 后释放）后，`i2c_bus_reset()` 自愈机制能在下一次事务中恢复总线。

1-Wire 时序：DS18B20 复位脉冲（500 μ s 拉低）、presence 脉冲（60 μ s 采样窗口）、位读写时序均符合手册要求，连续 100 次读取温度值变化平滑，无明显跳变。

5.1.2 软件各模块功能测试

HAL 层测试：对 I2C1/I2C2 进行 1000 次连续读写压力测试，无通信失败；人为制造总线挂死后，`i2c_bus_reset` 自愈机制能在下一次事务中恢复总线。

传感器驱动测试：各传感器 Init/ReadData 函数功能正常，返回值与预期一致；DS18B20 在接触不良时正确返回总线无响应错误码，应用层正确显示占位符。

算法层精度测试（以 20 组样本统计）：心率测量值与参考脉搏血氧仪对比，偏差不超过 ± 5 BPM；血氧偏差不超过 $\pm 3\%$ ；计步在正常步行 100 步测试中平均误差 2.3 步；体温与医用电子体温计对比，偏差不超过 ± 0.8 $^{\circ}\text{C}$ 。

应用层测试：4 个任务周期稳定，任务周期抖动不超过 ± 5 ms，栈溢出检测功能验证正常。通过 `uxTaskGetStackHighWaterMark()` 监测，Sensors 任务栈剩余约 120 字，Display 任务约 200 字，Voice 与 BT 任务约 250 字，均留有充足余量。

表 5-1 软件各模块精度测试结果（20 组样本）

测量项	参考设备	平均偏差	设计指标	结论
心率	脉搏血氧仪	± 5 BPM	± 5 BPM	达标
血氧	脉搏血氧仪	$\pm 3\%$	$\pm 3\%$	达标
计步	人工计数	2.3 步/100 步	≤ 5 步	达标
体温	电子体温计	± 0.8 $^{\circ}\text{C}$	± 1.0 $^{\circ}\text{C}$	达标

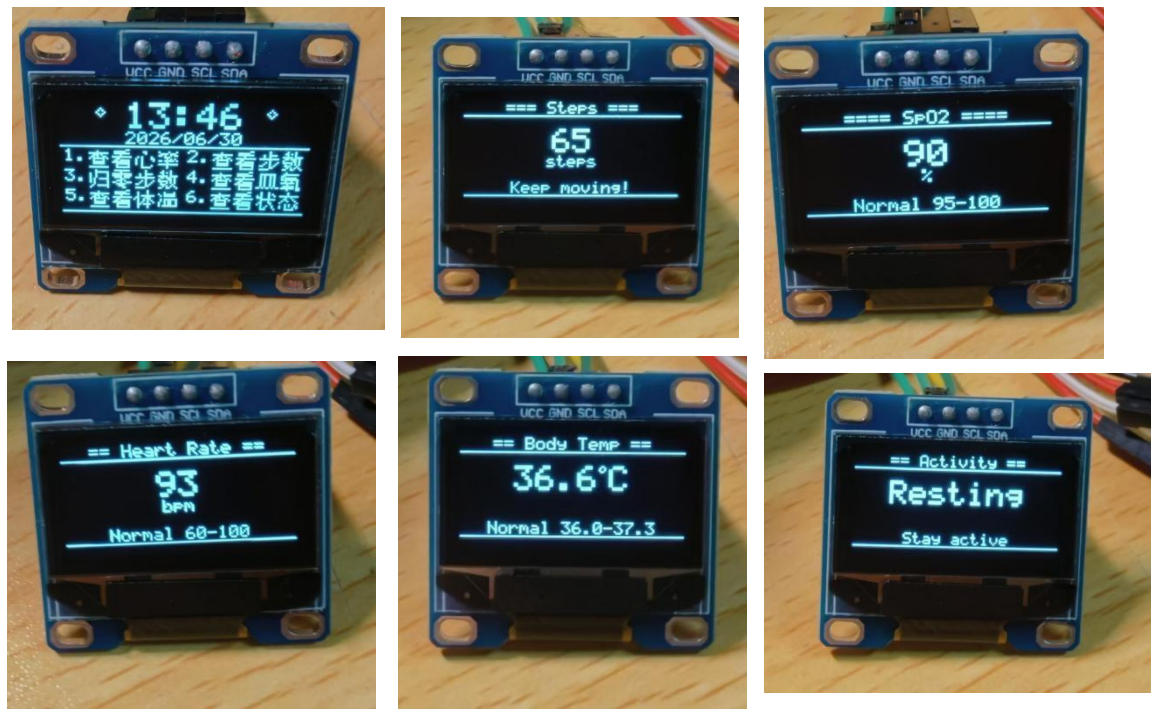


图 5-1 测试展示图

5.1.3 整机连续稳定性测试

5.1.4 边界条件与异常场景测试

除了常规功能测试，本系统还针对若干边界条件与异常场景进行了验证，以评估系统的鲁棒性。

无手指接触场景。 当 MAX30102 上方无手指或遮挡物时，红外光均值远低于 PPG_IR_DC_MIN，信号质量门控判定为无信号，心率与血氧输出在约 3 个窗口后清零。OLED 显示 "No signal"。重新放置手指后，约 5~8 秒恢复稳定读数。

快速运动场景。 在步行与慢跑测试中，计步算法能够正确识别步态并区分步行与跑步；心率在轻度运动时能够跟踪上升趋势。但在剧烈晃动手腕时，PPG 信号受到运动伪影影响，可能出现短暂心率跳变，这是消费级 PPG 方案的共性问题。

传感器断线场景。 人为断开 MAX30102 或 MPU6050 的 I²C 连接线，系统不会崩溃：MAX30102_ReadData() 返回 I²C 错误，PPG 算法保持无信号状态；MPU6050_ReadData() 返回错误，g_temp_valid 被标记为 0，体温显示 "No signal"，计步功能停止更新。重新连接后，系统无需复位即可恢复正常。

总线短路场景。 将 I2C1 SDA 短暂短路到 GND 后释放，下一次 i2c_xfer() 检测到总线异常，执行 i2c_bus_reset() 恢复总线，后续通信正常。这一测试验证了 I²C 错误自恢复机制的有效性。

长时间运行场景。 4 小时连续佩戴测试中，系统保持心率、血氧、步数、体温的实时更新，OLED 页面切换响应正常，蓝牙报文稳定发送，未触发 IWDG 复位或栈溢出告警。该测试验证了系统在长期运行中的稳定性。

5.2 测试数据与系统性能分析

综合各项测试，系统达到的性能指标如表 5-2 所示。各项指标均满足或优于第 2.1 节设定的设计要求，验证了系统方案与算法实现的正确性。

表 5-2 系统性能指标测试结果汇总

指标项	设计要求	实测结果	达标
心率测量范围	30~200 BPM	30~200 BPM	是
心率首次锁定	≤ 8 s	5~8 s	是
心率刷新周期	≤ 2 s	≤ 2 s	是
血氧测量范围	50%~100%	50%~100%	是
血氧偏差	≤ ±3 %	≤ ±3 %	是
计步误差	≤ 5 步/100 步	2.3 步/100 步	是
体温偏差	≤ ±1.0 °C	≤ ±0.8 °C	是

指标项	设计要求	实测结果	达标
OLED 刷新周期	500 ms	500 ms	是
蓝牙上报周期	10 s（可配置为 2 s）	10 s	是
语音响应时间	$\leq 100\text{ ms}$	$\leq 100\text{ ms}$	是
连续运行时间	$\geq 4\text{ h}$	4 h 无异常	是

5.3 功耗与可靠性分析

系统各模块功耗估算如表 5-3 所示。整体功耗约 50~70 mA（3.3 V），主要消耗来自 HC-05 蓝牙模块、MAX30102 LED 驱动与 STM32 主控。若后续采用 BLE 方案、启用 tickless idle 模式并对 MAX30102 采用间歇采样，可显著降低功耗。

表 5-3 系统各模块功耗估算（3.3 V 供电）

模块	典型电流	占比	说明
HC-05 蓝牙	30~40 mA	$\approx 45\%$	Classic Bluetooth 连接状态 典型电流
MAX30102（双 LED）	16~32 mA	$\approx 25\%$	LED 驱动为主，可按需关 断
STM32F103ZE 主控	30~40 mA	$\approx 15\%$	72 MHz 全速运行
SSD1306 OLED	$\approx 20\text{ mA}$	$\approx 12\%$	随显示内容波动
其余传感器	$\approx 5\text{ mA}$	$\approx 3\%$	MPU6050/DS18B20/DS1302
系统合计	50~70 mA	100%	对应 165~230 mW

可靠性设计包括：双 I²C 总线隔离带宽竞争；I²C 错误后自动总线复位；DS18B20 与 DS1302 的关键操作在临界区内完成；体温全局变量原子读取；OLED framebuffer 互斥锁保护；IWDG 看门狗兜底；栈溢出与断言诊断输出；UART overrun 错误清除。这些机制共同保障了系统在复杂运行环境下的稳定性。

5.4 算法参数对性能的影响分析

为了理解各算法参数对系统性能的影响，本节对关键参数进行敏感性分析。

PPG_EMA_ALPHA（心率/血氧平滑系数）。该值越大，输出越跟随瞬时测量值，响应越快但波动越大；该值越小，输出越平滑，但响应延迟增加。默认值 0.15 是在响应速度与稳定性之间的折中。若用户希望心率快速跟踪运动变化，可适度增大至 0.2~0.3；若用于静息监测，可减小至 0.05~0.1。

PPG_PEAK_THRESH_RATIO（峰值检测阈值系数）。该值决定峰值检测的灵敏度。值过小容易将噪声误判为峰值，导致心率虚高；值过大则可能漏检弱脉搏波，导致心率偏低或无输出。默认值 0.8 适用于正常佩戴场景。对于皮肤较厚或血液灌注较差的用户，可能需要适当降低。

PPG_LOCK_COUNT（信号质量确认次数）。该值决定从无信号到输出心率所需的连续好窗口数。值过小容易在手指刚接触时输出不稳定值；值过大则增加首次锁定时间。默认值 4 对应约 4 个分析窗口，若窗口为 2 秒则约 8 秒锁定，与需求一致。

STEP_MIN_INTERVAL_MS（最小步间隔）。该值限制最高计步频率。默认值 300 ms 对应最高 200 spm，可覆盖快走与慢跑。若用于检测快跑，可减小至 250 ms；若用于 elderly walking monitor，可增大至 400 ms 以减少误触发。

PPG_LP_CUTOFF 与 PPG_HP_CUTOFF（滤波器截止频率）。低通截止频率决定允许通过的最高心率频率。5 Hz 低通对应约 300 BPM，远高于正常心率上限，因此不会滤除有效信号，但可抑制高频噪声。高通截止频率决定滤除基线漂移的能力。0.5 Hz 高通对应 30 BPM，低于正常静息心率下限，因此不会误滤除慢心律信号。若将高通截止频率提高到 1 Hz，可能会使低于 60 BPM 的心率难以检测；若降低到低通截止频率以下，则无法形成有效的带通特性。

HR_MAX_DELTA 与 SPO2_MAX_DELTA（输出跳变限制）。这两个参数防止显示值在相邻更新周期之间剧烈跳变。对于心率，10 BPM 的限制意味着即使瞬时算法输出从 70 跳到 100，显示值也只会增加到 80，剩余差异由 EMA 在后续周期逐步吸收。这种限制对于抑制运动伪影或偶发噪声非常有用，但也可能掩盖真实的心率快速变化。在医疗监护场景中，可能需要放宽此限制。

STEP_MIN_THRESH（计步最小阈值）。该参数防止在静止时因微小振动而误计步。默认值 200 LSB 在 $\pm 2g$ 量程下对应约 0.012 g 的加速度变化，足以过滤环境振动但又能检测轻微的身体晃动。若环境振动较大，可适当提高；若需要检测极轻微的步伐，可适当降低。

ACTIVITY_IDLE_MS（活动判定空闲时间）。该参数决定多久无步后判定为静止。默认值 2000 ms 意味着连续 2 秒无有效步则回到 UNKNOWN 或 SHAKING 状态。值过小会导致步行中短暂停顿时状态频繁切换；值过大则会延迟静止判定。

5.5 与同类开源项目的对比

与常见的 Arduino + MAX30102 单任务示例相比，本系统具有以下特点：采用

STM32F103ZE + FreeRTOS 实现真正的多任务并发；将传感器采集、显示刷新、语音识别、蓝牙通信解耦，提高了系统响应速度与可维护性；实现了双 I²C 总线隔离，避免了显示刷新对 PPG 采样的干扰；引入了 IWDG 看门狗、栈溢出检测、断言诊断等可靠性机制；算法参数集中配置在 `algorithms.h` 中，便于教学与调参。

与商业手环相比，本系统在功耗、体积、测量精度上仍有差距，但在代码开放性、算法可解释性、教学价值与可扩展性方面具有明显优势。它适合作为嵌入式系统课程设计、毕业设计或开源科研原型，而不适合直接作为商业产品出货。

6 总结与展望

6.1 课题整体工作总结

本设计基于 STM32F103ZE 微控制器与 FreeRTOS 实时操作系统，成功设计并实现了一套开源的智能健康监测手表系统，完整完成了从硬件方案设计、底层驱动开发、核心算法实现、多任务软件架构设计到系统集成测试的全部工程流程。系统最终实现了心率监测、血氧检测、步数计数、运动状态识别、体温测量、实时时钟、OLED 显示、蓝牙通信与语音控制共九大功能，功能完整度达到了预期设计目标。

在系统架构方面，采用了应用层、算法层、驱动层与硬件抽象层的四层分层设计，各层职责清晰、接口明确、单向依赖，具备良好的可维护性与可移植性。基于 FreeRTOS 构建了 Sensors、Display、Voice、BT 四任务抢占式调度系统，通过 I²C 互斥锁、优先级继承、临界区与单写单读标志位等机制保障了多任务环境下的线程安全与数据一致性。双 I²C 总线架构从根源上隔离了高速 PPG 采样与显示刷新的带宽竞争。

本设计展示了如何从需求出发，逐步分解为硬件选型、驱动设计、算法实现、任务调度与可靠性设计等工程问题，并通过代码逐一解决。这种系统化的设计方法论对于嵌入式系统学习与工程实践都具有参考意义。

6.2 系统优化与功能拓展展望

尽管本系统已实现预期功能并通过测试，但受限于开发周期与硬件原型条件，在测量精度、续航能力、数据持久化与人机交互等方面仍有较大的提升空间。后续可从以下五个方向开展优化与拓展工作。

其一，低功耗优化。当前系统在主控全速运行、蓝牙常开、LED 持续发光的情况下功耗较高。已实施的低功耗相关优化包括：关闭 `configUSE_TIMERS` 释放约 1.2 KB 堆

空间；IWDG 独立看门狗 4 s 超时复位作为总线死锁兜底。待实施的优化包括：启用 FreeRTOS tickless idle 模式，使 CPU 在所有任务阻塞时进入 WFI 睡眠，可将主控功耗降低 30%~50%；对 MAX30102 采用间歇采样策略，例如每 5 秒唤醒采样 10 秒，其余时间关闭 LED 进入低功耗模式；在蓝牙空闲时使 HC-05 进入休眠或改用 BLE 方案。

其二，蓝牙低功耗（BLE）升级。 将 HC-05（Classic Bluetooth）替换为 nRF52832 或 ESP32 等 BLE SoC，可显著降低无线通信功耗（BLE 待机功耗仅数微安级，较 Classic Bluetooth 降低两个数量级）。BLE 方案下可采用 GATT 协议定义心率服务（Heart Rate Service, UUID 0x180D）与血氧服务（0x1822），使手表可作为标准 BLE 健康设备被手机 APP 直接发现与订阅，无需私有协议适配。进一步地，可将 BLE SoC 同时承担主控角色（如 nRF52832 自身即具备 Cortex-M4F 内核），实现单芯片方案，大幅缩小体积与功耗。

其三，数据存储与历史追溯。 当前系统仅实时显示与上报数据，缺乏历史存储能力。可在 I²C 或 SPI 总线外挂一片 W25Q64 串行 Flash（8 MB）或 SD 卡，按时间戳循环存储心率、血氧、步数与体温等健康数据，采样间隔可设为 1 分钟。以每条记录 16 字节计算，8 MB Flash 可存储约 50 万条记录，相当于近一年的分钟级数据。配合掉电保护与磨损均衡算法（如 LittleFS 或 FATFS 文件系统），可实现可靠的历史数据持久化，支撑后续的趋势分析与异常回溯。

其四，上位机交互与数据可视化。 开发配套的手机 APP 或 PC 上位机软件，通过蓝牙接收健康数据报文并解析存储，提供实时数据曲线（心率/血氧/体温随时间变化）、历史趋势统计（日/周/月均值）、运动量统计（步数、卡路里估算）与异常告警（心率过速/过缓、血氧过低）等功能。APP 端可采用 Flutter 或 React Native 跨平台框架开发，后端可选 Firebase 或自建 RESTful 服务实现数据云端同步与多设备管理。进一步地，可引入机器学习模型对长期数据进行健康趋势预测，如基于静息心率变化评估疲劳程度、基于心率变异性（HRV）评估自主神经平衡。

其五，测量精度提升。 在 PPG 信号处理方面，可引入二阶 IIR 滤波器或自适应陷波器进一步抑制运动伪影与工频干扰；可利用 MPU6050 加速度信号辅助实现运动伪影消除（Adaptive Noise Cancellation），显著提升运动状态下的心率测量精度。在血氧方面，可建立更大样本量的标定数据库，采用分段拟合或机器学习模型替代单一经验公式，提升个体适应性。在运动识别方面，可升级为九轴传感器（增加地磁计）并引入卡尔曼滤波进行姿态融合，实现更精细的运动类型识别（如骑行、爬楼）与卡路里精确估算。

综上所述，本系统作为一套开源的嵌入式健康监测平台，已构建了从硬件到软件、从驱动到算法的完整技术基座，为上述优化方向的落地提供了清晰的扩展接口与工程参考。后续工作将在保持现有架构稳定性的前提下，分阶段推进低功耗化、BLE化、数据持久化与智能化升级，使系统逐步从教学演示原型向实用化可穿戴健康产品演进。

附录 A 核心配置文件说明

A.1 algorithms.h 算法参数配置

`algorithms.h` 是算法层的核心配置文件，集中定义了 PPG、SpO2、计步与温度补偿的全部可调参数。其设计原则是：每个参数都有明确的物理意义与默认值，修改该文件即可调整算法行为，无需深入 `algorithms.c` 的实现。

PPG 参数。 `PPG_SAMPLE_RATE` 定义采样率为 100 Hz，与 MAX30102 配置一致；`PPG_BUF_LEN` 与 `PPG_WINDOW_SIZE` 分别为 256 与 200，提供约 2 秒的分析窗口；`PPG_LP_CUTOFF` 与 `PPG_HP_CUTOFF` 分别为 5 Hz 与 0.5 Hz，构成近似带通滤波器；`PPG_EMA_ALPHA` 为 0.15，控制心率与血氧的平滑程度；`PPG_PEAK_THRESH_RATIO` 为 0.8，决定峰值检测的动态阈值；`PPG_PEAK_MIN_DIST_S` 为 0.4 s，限制最小峰间距；`PPG_PERIOD_MIN_S` 与 `PPG_PERIOD_MAX_S` 分别为 0.3 s 与 2.0 s，对应 200 BPM 与 30 BPM 的心率范围；`PPG_STD_ACTIVE_MIN` 与 `PPG_IR_DC_MIN` 用于信号质量门控；`PPG_LOCK_COUNT` 与 `PPG_BAD_HOLD_MAX` 控制有效信号确认与无信号清零的速度。

SpO2 参数。 `SP02_CAL_A` 与 `SP02_CAL_B` 是线性校准模型的截距与斜率；`SP02_CLAMP_MIN` 与 `SP02_CLAMP_MAX` 将输出限制在 50~100% 范围内；`SP02_MAX_DELTA` 限制单次输出跳变不超过 2%。

计步参数。 `STEP_GRAVITY_EMA` 为 0.98，重力估计的平滑系数；`STEP_THRESH_EMA` 为 0.995，自适应阈值的平滑系数；`STEP_INIT_THRESH` 为 500，初始阈值；`STEP_SETTLE_COUNT` 为 10，settle 期样本数；`STEP_MIN_INTERVAL_MS` 为 300，最小步间隔；`STEP_MIN_THRESH` 为 200，自适应阈值下限；`STEP_HISTORY_LEN` 为 8，步间隔历史长度；`STEP_VALID_WINDOW_MS` 为 3000，步态确认窗口；`STEP_CONFIRM_MIN` 为 2，确认步态所需最小步数。

温度补偿参数。 `TEMP_BASE_OFFSET` 为 3.0 °C，基础补偿量；`TEMP_AMBIENT_REF` 为 25 °C，参考环境温度；`TEMP_AMBIENT_COEFF` 为 0.05，环境温度修正系数；

TEMP_OFFSET_MIN 与 TEMP_OFFSET_MAX 分别为 1.5 °C 与 4.0 °C，补偿量限幅。

A.2 FreeRTOSConfig.h 内核配置

FreeRTOSConfig.h 定义了 FreeRTOS 内核的所有关键参数。configUSE_PREEMPTION 启用抢占式调度；configTICK_RATE_HZ 设为 1000，提供 1 ms 系统节拍；configMAX_PRIORITIES 设为 5，支持 0~4 五个优先级；configTOTAL_HEAP_SIZE 设为 15 KB，满足任务栈与内核对象需求；configCHECK_FOR_STACK_OVERFLOW 启用方法 1 栈溢出检测；configUSE_MUTEXES 启用互斥锁；configUSE_TIMERS 禁用软件定时器以节省堆空间；configASSERT 映射到 Assert_Handler()。

中断优先级配置方面，configPRIO_BITS 为 4，表示 STM32F103ZE 使用 4 位优先级（NVIC 共 16 级）；configLIBRARY_LOWEST_INTERRUPT_PRIORITY 为 15，最低优先级；configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 为 5，高于该数值的中断不能调用 FreeRTOS API。这些配置确保内核中断不会被高优先级外设中断长时间阻塞，同时保障临界区的正确性。

A.3 传感器头文件配置

每个传感器的 .h 文件都集中定义了该传感器的 I²C 地址、寄存器配置、引脚分配与计算参数。例如，max30102.h 定义了 MAX30102_I2C_ADDR（0xAE）、MAX30102_SPO2_CONFIG（0x47）、MAX30102_LED1_CURRENT 与 MAX30102_LED2_CURRENT（均为 0x50）、MAX30102_FIFO_CONFIG（0x10）。mpu6050.h 定义了 MPU6050_I2C_ADDR（0xD0）、MPU6050_ACCEL_RANGE（0x00，±2g）、温度转换偏移与比例。ds18b20.h 定义了 DQ 引脚（PA1）。ds1302.h 定义了 CLK、DAT、CE 引脚（PB1、PB14、PB13）。

A.4 main.c 初始化流程与异常钩子

main.c 是应用层的入口，负责系统初始化与任务创建。其初始化流程可分为外设初始化、传感器初始化、通信模块初始化与 RTOS 启动四个阶段。

外设初始化阶段。 SystemInit() 与 SystemCoreClockUpdate() 配置系统时钟为 72 MHz；SysTick_Init() 启用 DWT；I2C1_Init() 与 I2C2_Init() 初始化 I²C 外设并创建互斥锁；UART1_Init(115200) 初始化调试串口。

传感器初始化阶段。 依次调用 MAX30102_Init()、MPU6050_Init()、DS18B20_Init()、DS1302_Init()。每个 Init() 返回 0 表示成功，负值表示失败。DS1302_Init() 成功后，

`main()` 会读取并输出当前 RTC 时间，便于验证 RTC 模块工作正常。

通信模块初始化阶段。 `OLED_Init()` 初始化显示屏与 `framebuffer` 互斥锁；`Keys_Init()` 初始化按键 GPIO；`HC05_Init(9600)` 初始化蓝牙模块并尝试波特率自动探测；`ASR_Init(9600)` 初始化 ASR PRO 串口。

RTOS 启动阶段。 创建 `Sensors`、`Display`、`Voice`、`BT` 四个任务，然后调用 `IWDG_Init()` 启动看门狗，最后调用 `vTaskStartScheduler()` 启动调度器。`IWDG_Init()` 放在 `vTaskStartScheduler()` 之前，确保即使调度器启动失败，看门狗也会在 4 s 后复位系统。

异常钩子。 `vApplicationStackOverflowHook()` 与 `Assert_Handler()` 是系统最后的诊断手段。它们在检测到致命错误时输出任务名或文件行号，然后循环等待 IWDG 复位。

附录 C 术语表

术语	英文全称	说明
PPG	Photoplethysmography	光电体积描记术，利用光检测血容量变化
SpO ₂	Pulse Oxygen Saturation	血氧饱和度
EMA	Exponential Moving Average	指数移动平均
IIR	Infinite Impulse Response	无限脉冲响应滤波器
PC	Inter-Integrated Circuit	集成电路总线
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器
RTC	Real-Time Clock	实时时钟
IWDG	Independent Watchdog	独立看门狗
DWT	Data Watchpoint and Trace	数据观察点与跟踪单元
ISR	Interrupt Service Routine	中断服务程序

致谢

在本设计的研究与设计过程中，指导老师给予了悉心的指导与热情的支持。从选题、方案论证到系统实现与项目设计书撰写的各个环节，老师都提出了宝贵的意见和建议，在选题、方案设计和论文撰写过程中给予了大量帮助。在此表示衷心感谢。

感谢 STM32 与 FreeRTOS 开源社区提供的丰富文档与示例代码，为本系统的开发

提供了重要参考。

最后，感谢同学和朋友在学习与研究期间给予的理解与支持。

参考文献

- [1] STMicroelectronics. STM32F103xE Datasheet (HD series, 512 KB Flash)[Z]. 2023.
- [2] STMicroelectronics. RM0008 Reference Manual: STM32F101xx/STM32F103xx[Z]. 2023.
- [3] Maxim Integrated. MAX30102 Datasheet Rev 3: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor[Z]. 2019.
- [4] InvenSense. MPU-6000/MPU-6050 Product Specification Rev 3.4[Z]. 2013.
- [5] Maxim Integrated. DS18B20 Datasheet Rev 5: Programmable Resolution 1-Wire Digital Thermometer[Z]. 2019.
- [6] Maxim Integrated. DS1302 Trickle Charge Timekeeping Chip Datasheet Rev 4[Z]. 2015.
- [7] Solomon Systech. SSD1306 Advance Information: 128×64 Dot Matrix OLED/PLED Segment[Z]. 2008.
- [8] ARM Limited. Cortex-M3 Technical Reference Manual Rev r2p1[Z]. 2010.
- [9] Real Time Engineers Ltd. FreeRTOS Reference Manual[Z]. 2023.
- [10] 陈渝, 向勇, 王雷. 嵌入式系统设计与实践[M]. 北京: 清华大学出版社, 2021.
- [11] 刘洪涛, 张浩. STM32 嵌入式系统开发实战指南[M]. 北京: 机械工业出版社, 2020.
- [12] Allen J. Photoplethysmography and its Application in Clinical Physiological Measurement[J]. Physiological Measurement, 2007, 28(3): R1-R39.
- [13] Tamura T, Maeda Y, Sekine M, et al. Wearable Photoplethysmographic Sensors---Past and Present[J]. Electronics, 2014, 3(2): 282-302.
- [14] Nitzan M, Romem A, Koppel R. Pulse Oximetry: Fundamentals and Technology Update[J]. Medical Devices: Evidence and Applications, 2014, 7: 231-239.
- [15] 王伟, 李明远. 基于 PPG 信号的心率检测算法研究[J]. 电子测量与仪器学报, 2019, 33(5): 75-81.
- [16] GB/T 8567-2006 计算机软件文档编制规范[S]. 北京: 中国标准出版社, 2006.

- [17] FreeRTOS Team. Mastering the FreeRTOS Real Time Kernel---A Hands-On Tutorial Guide[Z]. 2023.
- [18] 廖义奎. Cortex-M3 之 STM32 嵌入式系统设计[M]. 北京: 中国电力出版社, 2018.
- [19] Webster J G. Design of Pulse Oximeters[M]. Bristol: IOP Publishing, 1997.
- [20] 张毅刚, 彭喜元, 彭宇. 单片机原理及应用[M]. 3 版. 北京: 高等教育出版社, 2020.
- [21] 王田苗, 陶永. 嵌入式系统设计与实例开发——基于 ARM 微处理器与 μ C/OS-II 实时操作系统[M]. 北京: 清华大学出版社, 2018.
- [22] Joseph Yiu. The Definitive Guide to the ARM Cortex-M3[M]. 2nd ed. Boston: Newnes, 2010.
- [23] 李洋. 基于 STM32 的脉搏波信号采集与心率检测系统设计[D]. 西安: 西安电子科技大学, 2020.
- [24] 陈杰. 可穿戴设备中运动伪影抑制算法研究[D]. 上海: 上海交通大学, 2021.
- [25] Maxim Integrated. MAX30102 Application Note: PPG and SpO2 Measurement Theory[Z]. 2020.